



需求文档

Requirements Documentation

李起成

College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



南开大学
Nankai University

软件需求规格说明

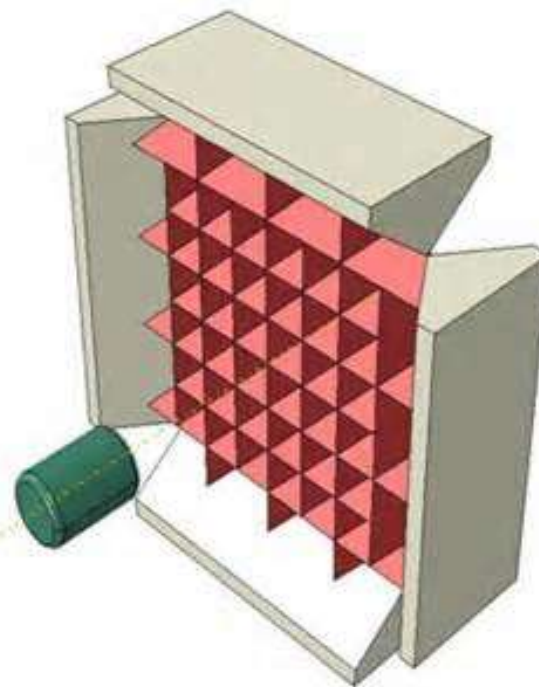
(Software Requirements Specification, SRS)

- 是具有一定法律效力的合同文档
- 清楚地描述软件在什么情况下，需要做什么，以及不能做什么
- 以输入/输出、输入到输出之间的转换方式来描述功能性需求
- 描述经过干系人磋商达成共识的非功能性需求
- 一般参考需求定义模板，覆盖标准模板中定义的所有条目
- 作为后续的软件评估依据和变更的基准



需求文档的组织形式

- 文档需要有逻辑组织结构
 - 例如：参照IEEE的模板
- 典型的组织形式包括
 - 按系统能够响应的各种外部环境情况来组织
 - 按系统特征来组织
 - 按系统的响应方式来组织
 - 按所管理的外部数据对象来组织
 - 按用户类型来组织
 - 按软件的工作模式来组织
 - 按子系统的划分来组织



软件需求规格说明 SRS的风格

- 描述性的自然语言文本

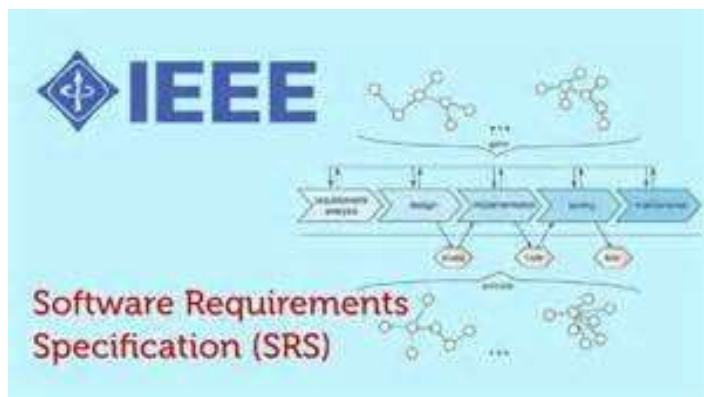
- 用户故事

- 从用例模型产生

- 用例模型与需求转化可看成可逆的过程
- 如果需求模型以用例的形式表示，我们可以逆向生成需求的完整集合

- 从需求数据库中生成

- 商业需求数据库有内置的功能来生成经过筛选的需求规格说明
- 从产品线需求规格数据库中生成特定产品的需求规格说明



选择合适的需求规格说明方式

- 考虑以下两个具体项目场景:

1) 小型项目, 1名程序员, 2个月的开发周期

程序员直接和用户对话, 写了两页纸的备忘录

2) 大型项目, 50 名程序员, 2年的开发周期

专门的团队进行需求建模与分析, 撰写了500页的需求规格说明

	项目 A	项目 B
需求撰写的目的	凝练工程师对问题的理解; 获取用户反馈	文档本身就是交付物; 为研发人员提供丰富细节
管理方面的用途	资源的分配与规格说明书无关; 相关资源已经就位	基于规格说明进行估算和任务进行规划
预期读者	主用户: 作者本人; 从用户: 客户	主用户: 程序员, 测试人员, 项目经理; 从用户: 客户



生成不同风格SRS的方法总览

方法	使用时机	适合	需要的资源
描述性文本	小项目	小产品的非正式说明	有良好写作能力的分析师
从用例模型生成	大中型项目	产品线的正式规格描述	有建模能力，好的过程标准，建模工具的分析师
从需求数据库生成	大中型项目	产品线的正式规格描述	需求数据库，有数据库管理能力的分析师

用户手册作为SRS

- 撰写用户手册作为一种性价比高的一箭双雕的方法，同时获得SRS和用户手册
- 用户手册作为SRS对于和用户交互的系统是比较有效的，这样系统由交互驱动
- 好的手册描述了所有用例的所有场景
 - Fred Brooks的《人月神话》中描述了将用户手册作为需求规格说明书的做法
- 据说在苹果的第一台电脑的编码工作开始之前用户手册就写好了，但是用户手册并没有描述：
 - 非功能性需求
 - 不和用户交互的功能性需求
 - 比如：函数计算、过滤器或者翻译工具的时候

高质量需求规格说明

需求规格说明要覆盖软件产品的功能、外部接口、性能、质量属性、设计约束！

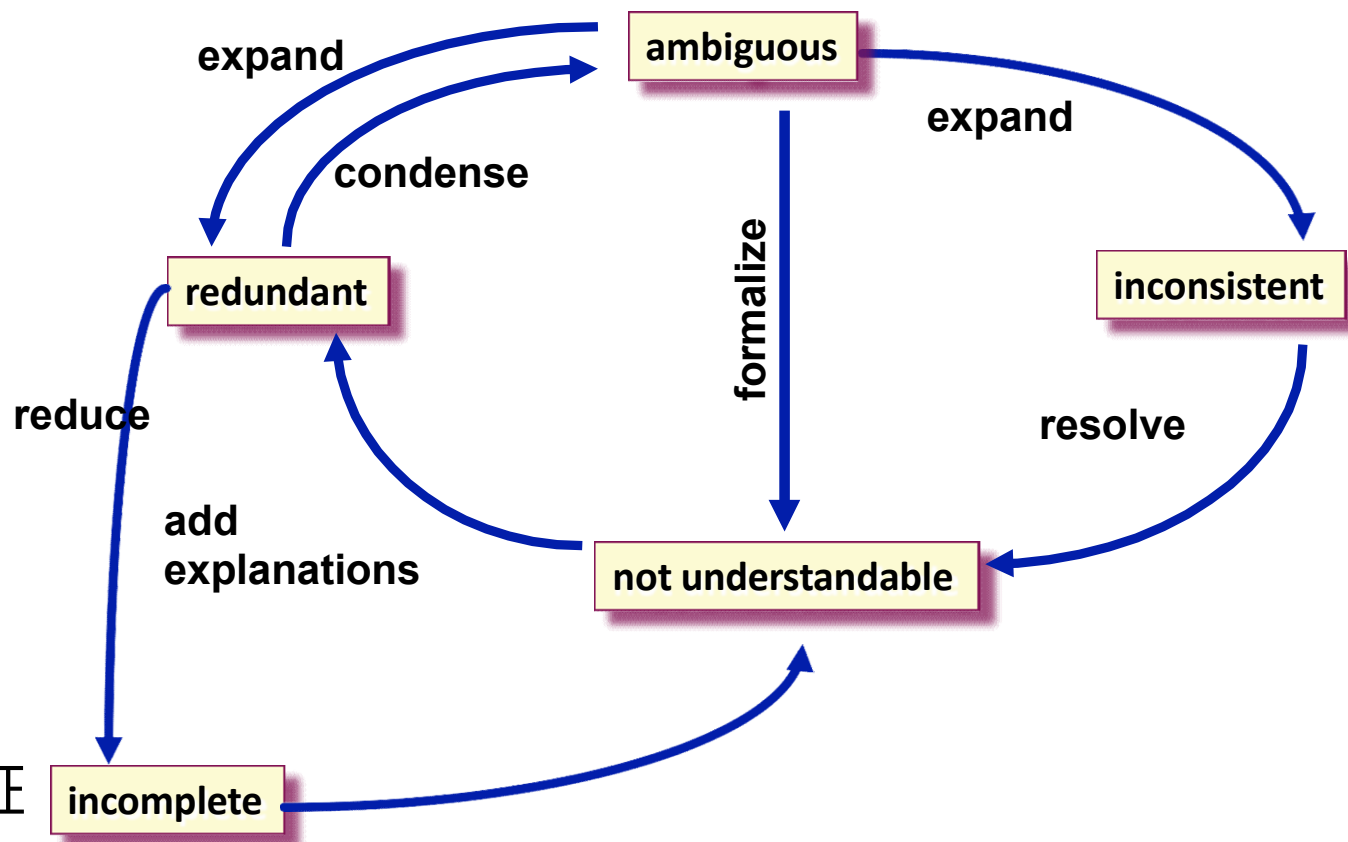
一个高质量的需求规格说明

- 是所有需求的集合
- 描述产品要提供的所有功能
- 是软件系统解决方案的商业合同的基础
- 是测试计划的基础
- 定义产品需求的度量标准
- 是产品需求跟踪的先决条件
- 影响开发产品的项目计划



高质量需求规格说明的评价标准

- 正确性 = 经过验证的
- 无歧义
- 完整的
- 可测试 = 可以证明的
- 可修改的
- 可跟踪的
- 易理解
- 一致的
- 有序的
- 项目或产品特定的其他特征



简洁

- 定义：需求描述尽可能简洁

- 描述了系统的一个独立特征
- 除了必需的信息外没有包含其他信息
- 用清晰、简单、可理解的词表述
- 避免“应该”、“可以”、“可能”之类的用词

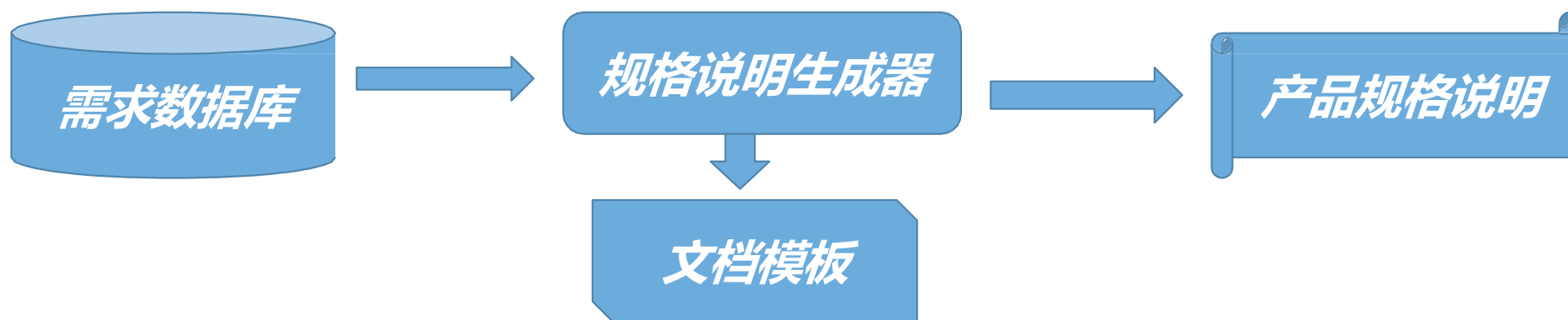


- 举例:

- “急救电话的响应应本着先到先响应的原则” 相对
- “急救电话应按照其拨入的次序存入一个先入先出的等待队列当中，并且按照进入队列的次序逐一应答” 是简洁的

需求规格说明生成过程

- 创建任何类型的需求规格说明最优先的方式是通过需求数据库生成它
- 不直接修改规格说明，修改数据库中的需求并且重新生成规格说明



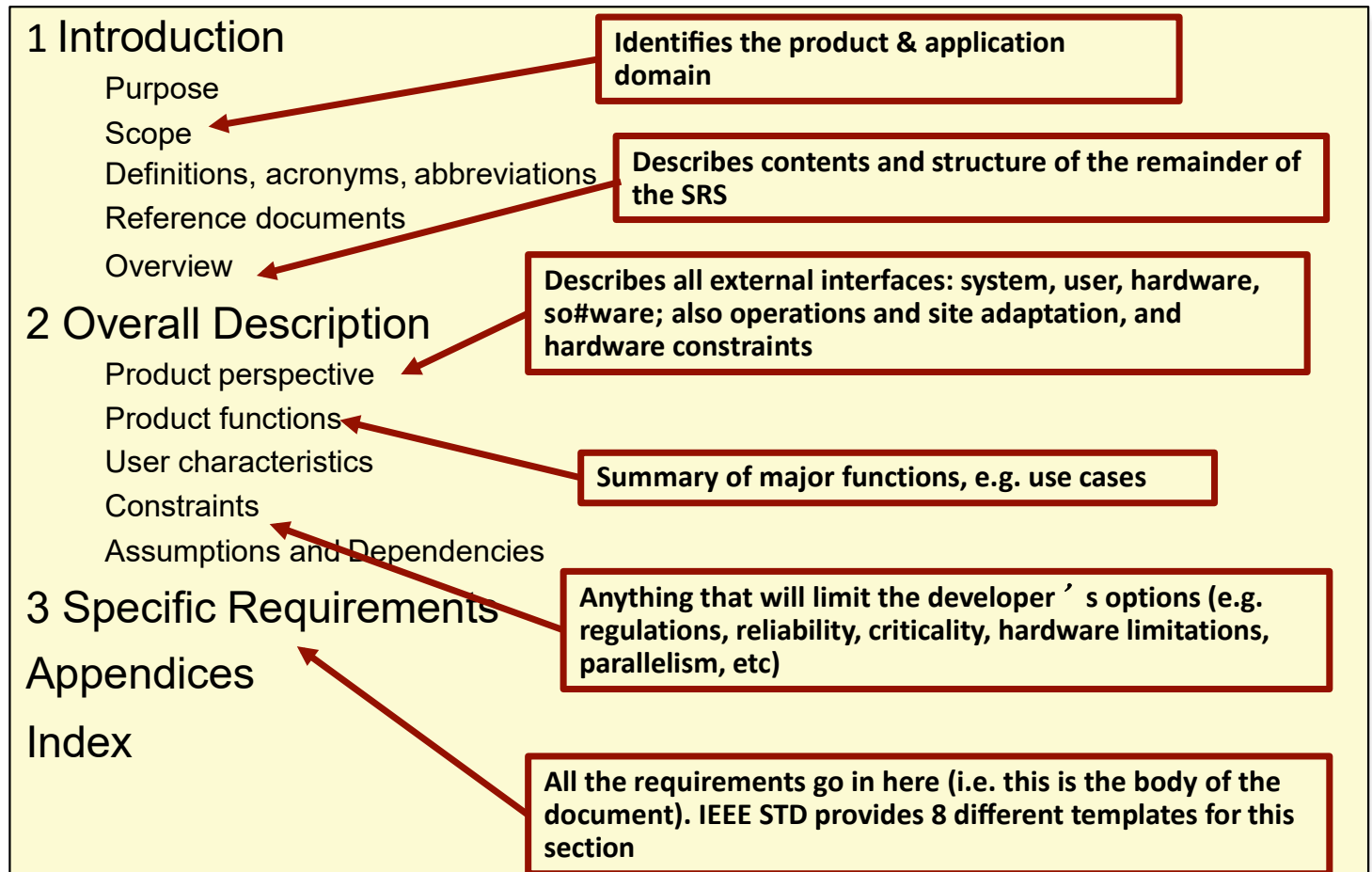
需求规格说明**SRS**模板

- SRS需要根据预先定义的标准章节模式来组织，即根据模板来撰写
- SRS的模板使得撰写统一的SRS变得简单
- 对于QA人员来说定义SRS指标变得简单
- 模板也适用于业务需求和系统需求
- 模板可以被用于半自动地从需求数据库或者用例模型生成SRS



IEEE-830 SRS模板大纲

- 介绍
- 术语表
- 文档总览
- 系统结构
- 系统主要功能
- 系统限制
- 系统功能和非功能描述
- 附录
- 索引



Source: Adapted from IEEE-STD-830-1993 See also, Blum 1992, p160



IEEE-830 SRS模板第三部分

1. 外部接口

1. 用户接口
2. 硬件接口
3. 软件接口
4. 通信接口

2. 功能需求描述

按系统的操作模式、用户分类、特征分类来定义:

1. 模式 1

1. 功能需求 1.1

...

2. 模式 2

1. 功能需求 2.1

...

...

3.2.n 模式 n

3. 性能需求

要用可度量的方式来表达!

4. 设计约束

1. 标准依从性

2. 硬件限制

etc.

5. 软件质量属性

1. 可靠性

2. 可用性

3. 安全性

4. 可维护性

5. 可移植性

6. 其他需求

Source: Adapted from IEEE-STD-830-1993. See also, Blum 1992, p160



SRS模板的优缺点

优点

- 模板提高效率
- 在有模板的情况下，面对一个完整的大纲，不容易遗漏重要的信息



缺点

- 并非对于所有的系统，模板的章节设计都是类似的
- 如果仅仅为了满足标准，而填写模板的所有章节，在不相关的章节，会加入一些没有意义的内容
- 读者很难将这些无意义的文字和真正的需求分开

总结

- 尽快开始写需求
- 产生一个初始版本来刺激反馈
- 询问用户往往比咨询专家更有用
- 撰写需求时需要遵循的法则：
 - 使用简单、直接的语言
 - 撰写可测试的需求
 - 使用定义好的并达成共识的术语



随时准备迎接需求变化

- 这是一种态度
- 越多的干系人参与，将获得越多的需求特征
- 但不能通过减少干系人的方法来解决这个问题
- 干系人有改变他们想法的权利
- 不要问：“这是你最终的需求吗？”
- 请将变化看成机会，而非威胁





软件系统建模与设计

李起成

College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



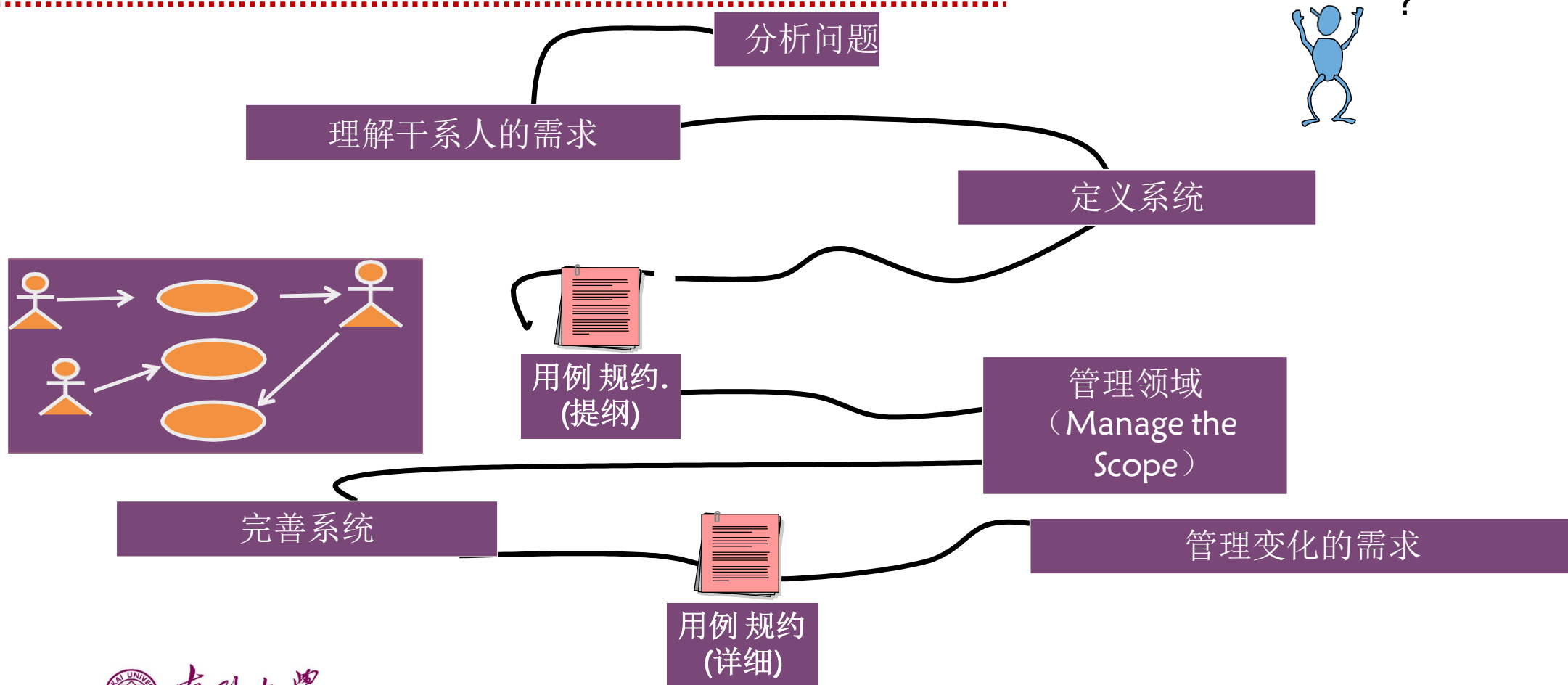
南开大学
Nankai University

情景驱动的需求建模 -- 用例建模

Use Case Modelling

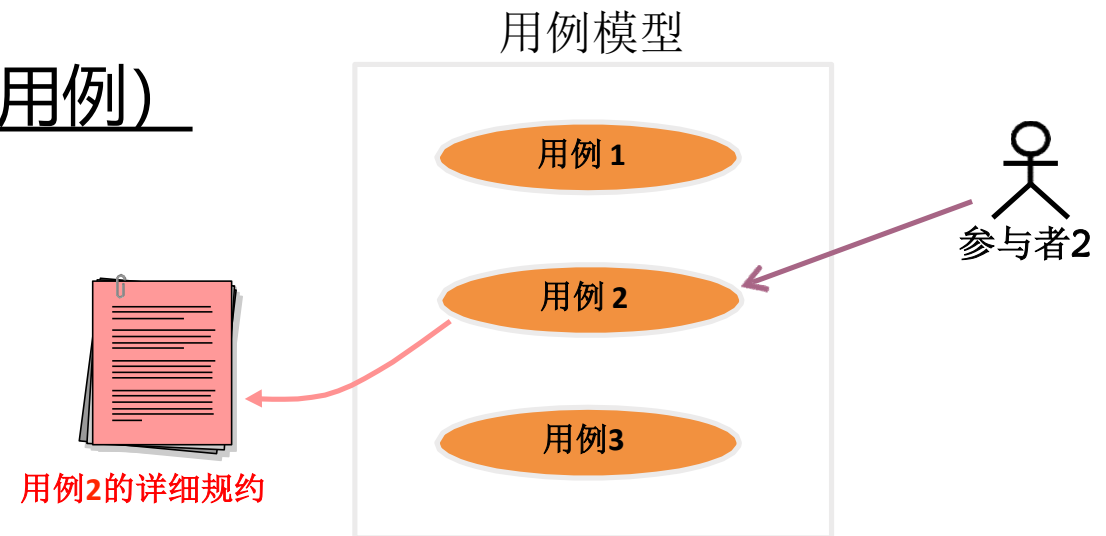


用例在需求管理过程中的作用

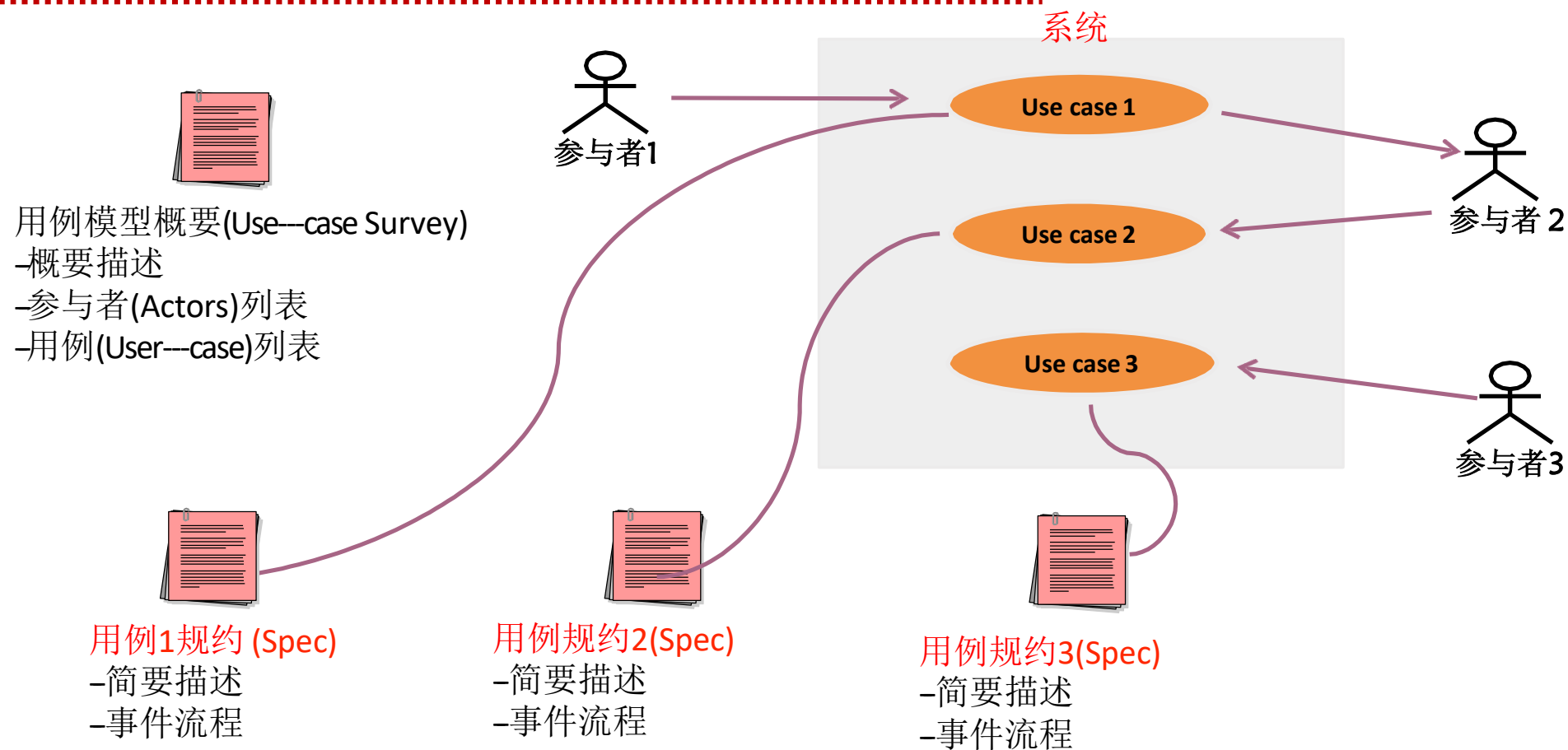


为什么需要用例建模——描述系统的功能性需求

- 关联于系人需要以及软件需求
- 确认与系统交互的人或对象（参与者）
- 定义系统的边界
- 捕捉和传达系统的理想行为（用例）
- 验证或确认需求
- 规划工具



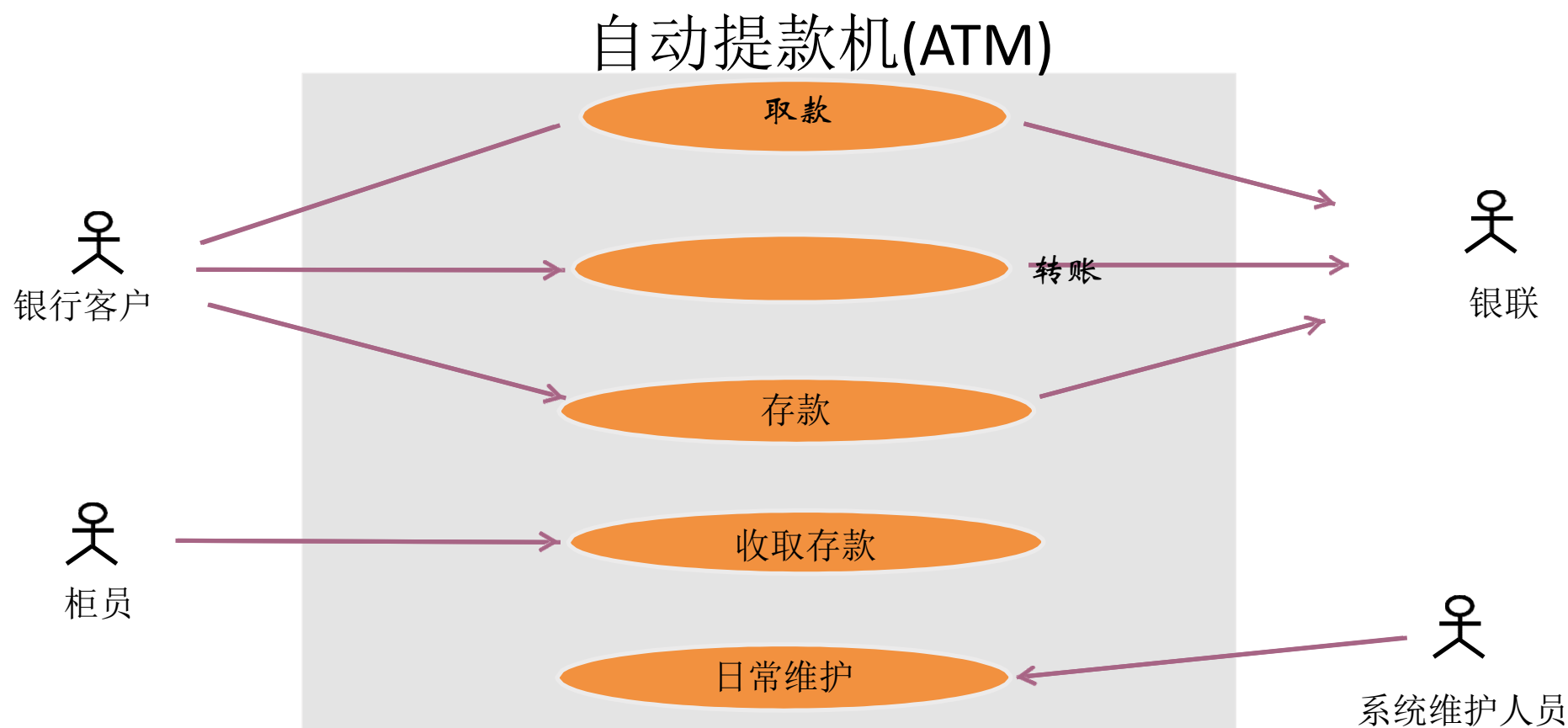
用例模型的表示——文本描述



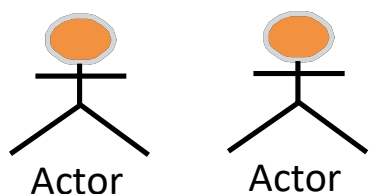
用例模型的表示——用例图

- ATM涉及哪些业务？
- ATM会与哪些系统或对象进行交互？
- 不同对象、系统是如何和ATM进行交互的？

用例模型的表示——用例图



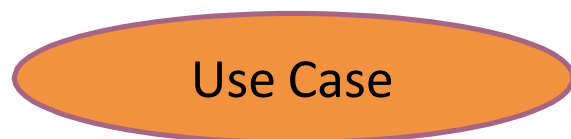
用例图的主要元素



参与者 (Actor)

与系统交互的人或外部系统

用例 (Use case)



系统为参与者提供的有价值的服务功能

关联 (Association)



用例图中用例与参与者之间的交互关系

什么是用例?



Use Case Name

一个用例

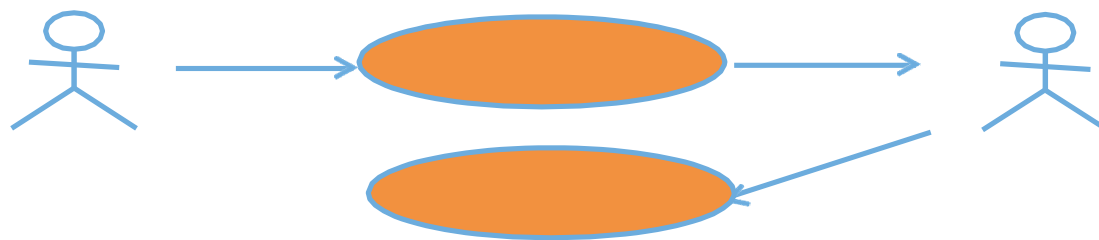
定义**系统**的一系列**行为**

通过此可为**参与者**提供**有价值**且**可观测**的结果。

粒度要适中

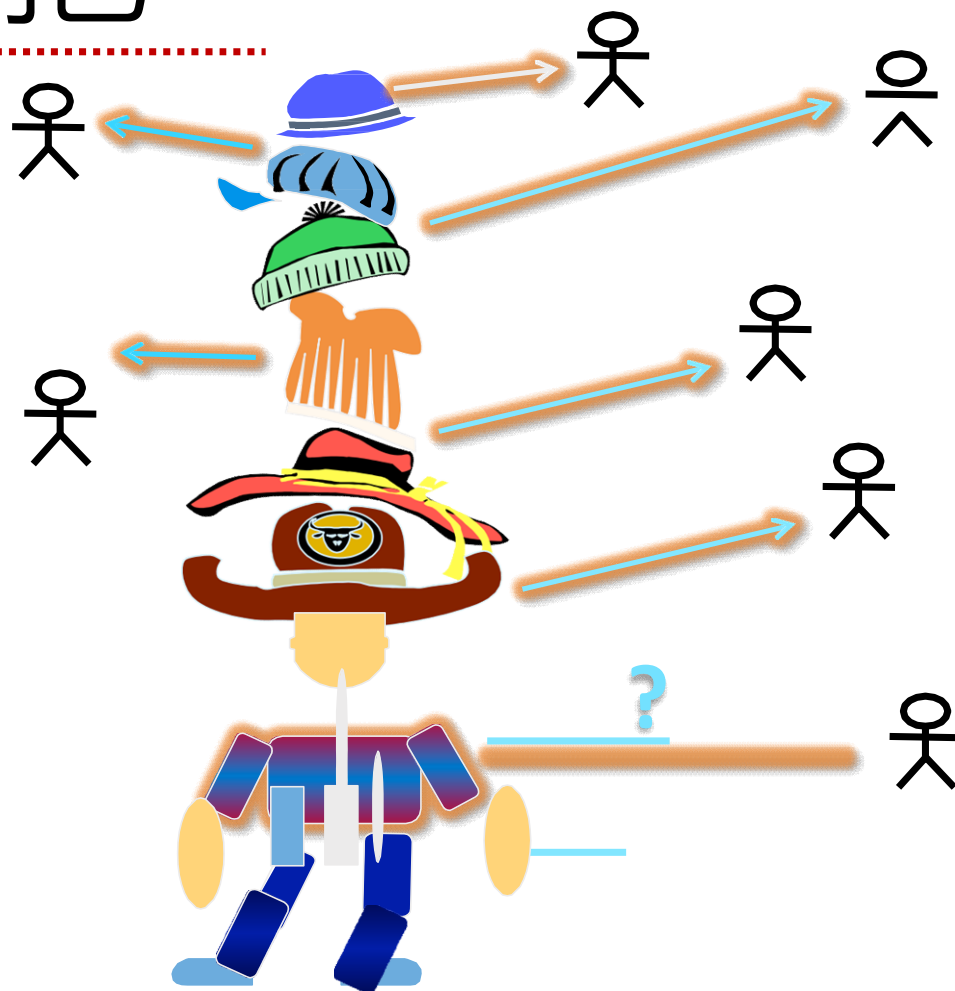
用例包含软件系统需求

- 用例
 - 定义一个参与者要用到的系统功能
 - 描述系统为实现参与者价值所开展的行为序列
 - 对参与者与系统之间的交互活动进行建模
 - 从特定的用户角度出发，是完整的，实现特定用户价值的事件流



参与者的定义：关注角色

- 与系统交互的人
- 与系统交互的硬件组件
- 或者其他的外部系统
- 关注的重点是所承担的“角色”
- 参与者的名要明确定义其角色



例：参与者定义与角色划分



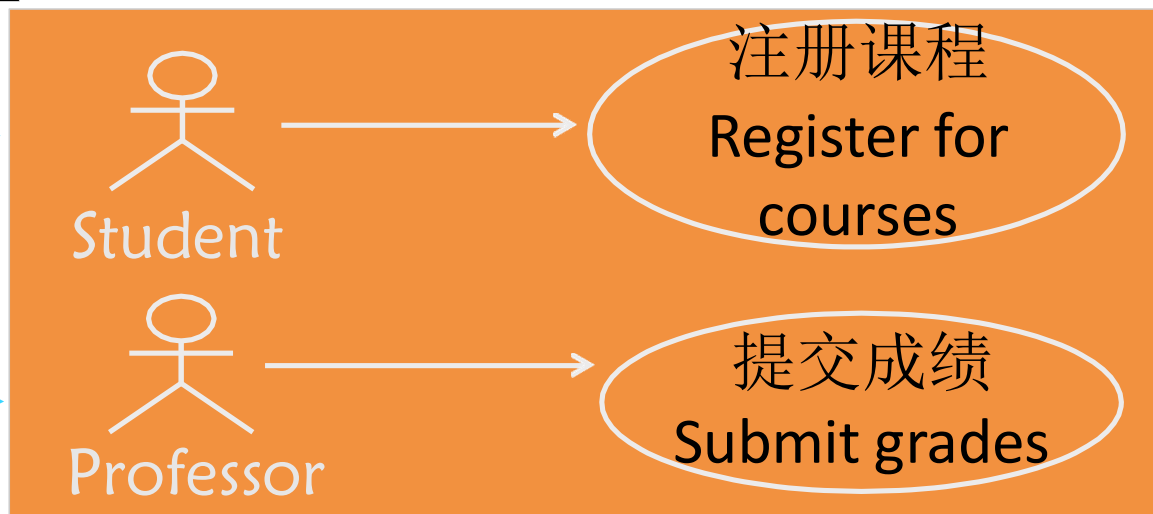
李雷：数学系的教授
经管学院的博士生



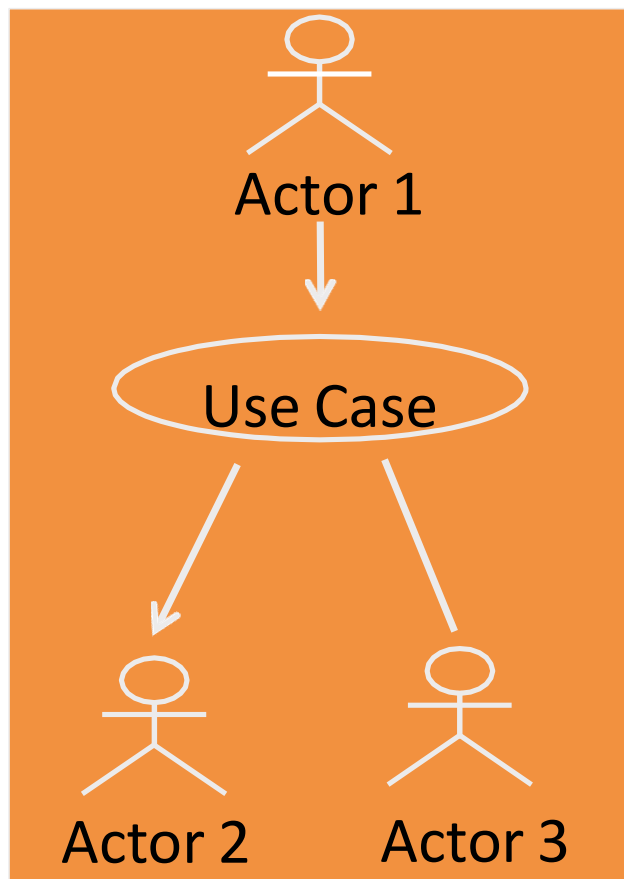
韩梅梅：软件学院本科生

李雷和韩梅梅
都具有学生角色

李雷同时也具有
教授的角色

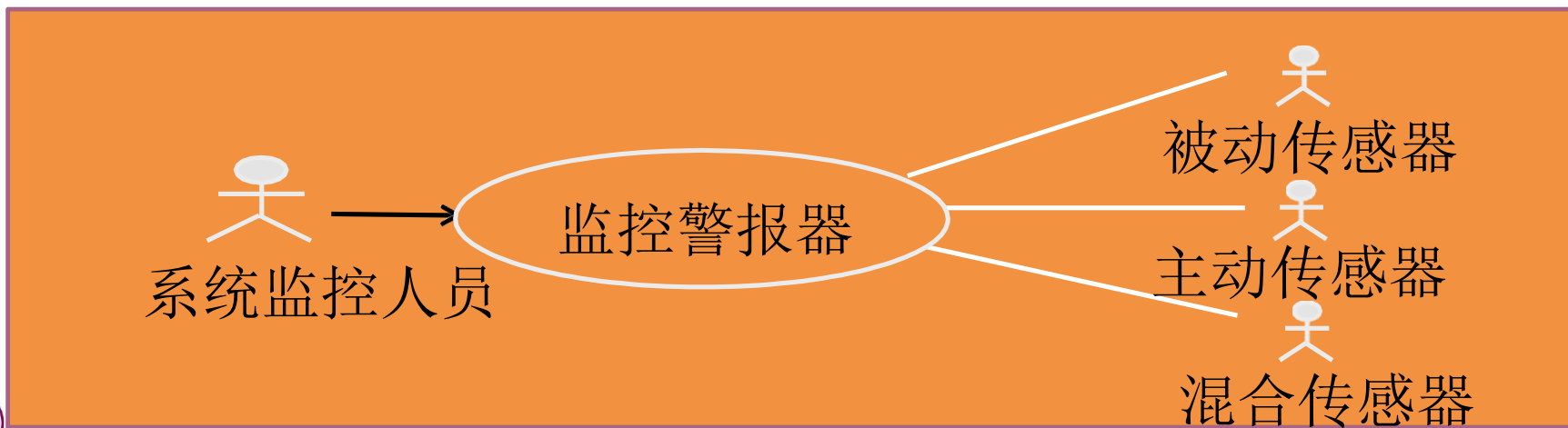
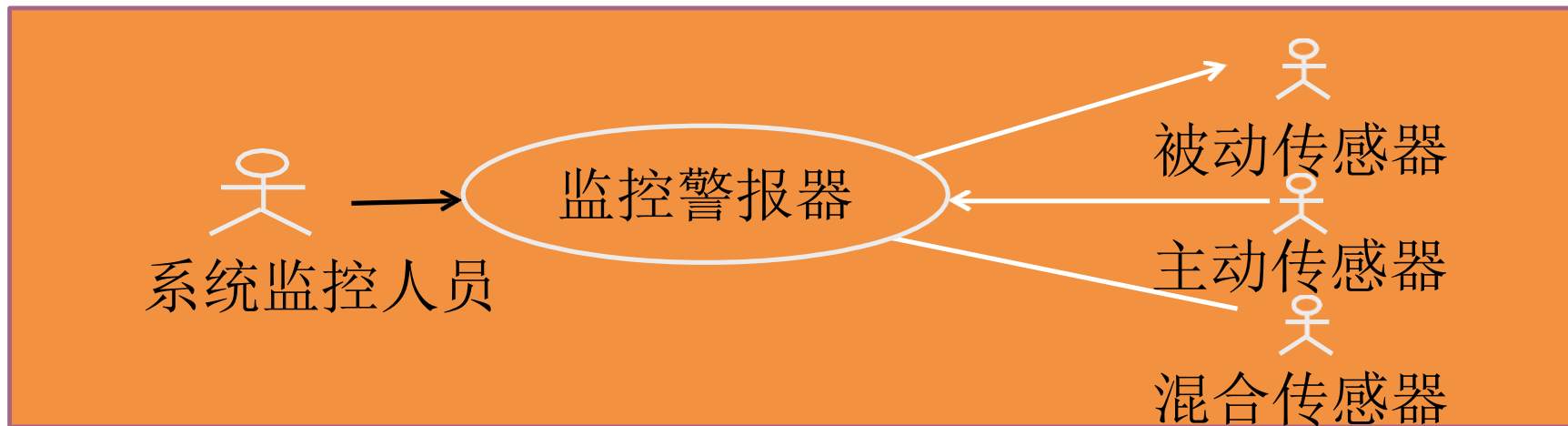


交互——关联 (Association)



- 参与者与用例之间的交互通道
- 用一条直线表示交互——关联
 - 有箭头的关联指出是谁发起的交互
 - 没有箭头则表明双方都可以发起交互

箭头的使用习惯



场景是用例的实例



- 场景1
- 登录系统 认证登录
- 输入主题词进行查询
- 获取课程列表 显示课程列表 选择课程
- 确认课程可选
- 显示最终的课程表

- 场景2
- 登录系统 认证登录
- 输入主题词进行查询 无效主题词
- 再次输入课程
- 获取课程列表 显示课程列表
- 选择课程
- 确认课程可选
- 显示最终的课程表

构建用例模型的步骤

- 第一步：找到所有的参与者和用例
 - 识别出参与者并做简单的描述
 - 识别出用例并做简单的介绍
- 第二步：编写用例
 - 列出用例
 - 给用例事件流程划分重要等级
 - 按照重要程度排序详细描述事件流程

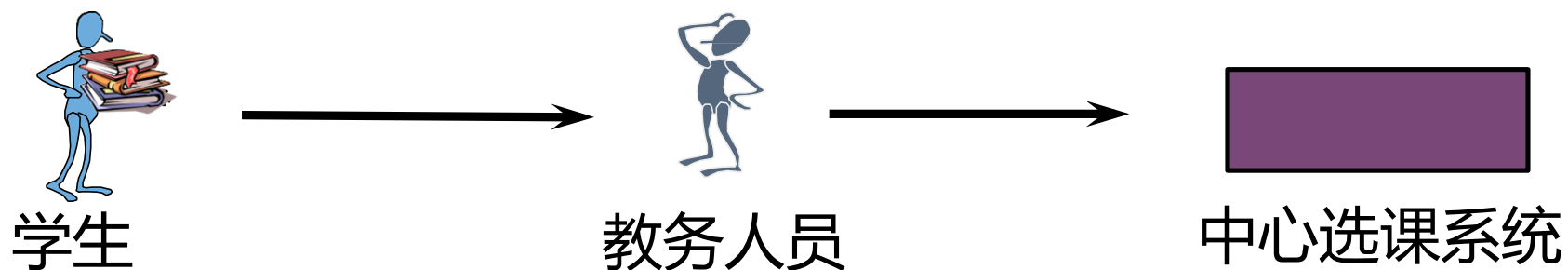


寻找参与者

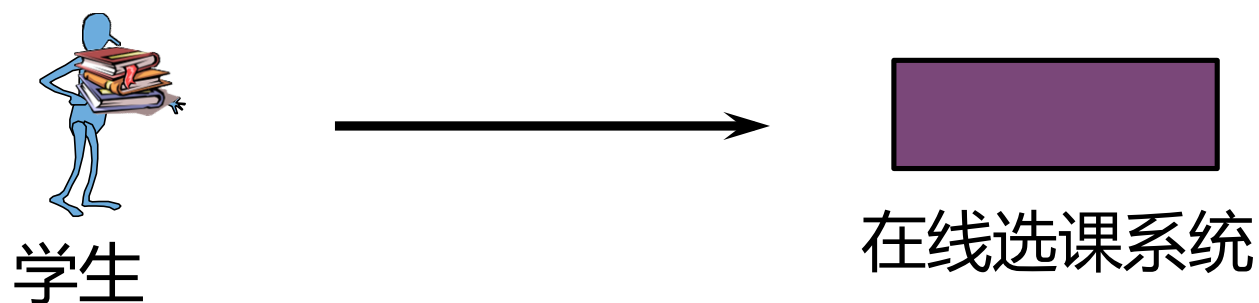
- 谁/什么使用系统?
- 谁/什么从系统中获取信息?
- 谁/什么向系统提供信息?
- 公司的哪个部门会使用系统?
- 谁/什么负责系统的维护?
- 还有哪些其他系统会使用系统?



识别参与者——是谁与系统进行交互？



学生并不直接操作选课系统；是教务人员进行操作。
或者，构建一个基于浏览器的在线应用？



参与者的描述

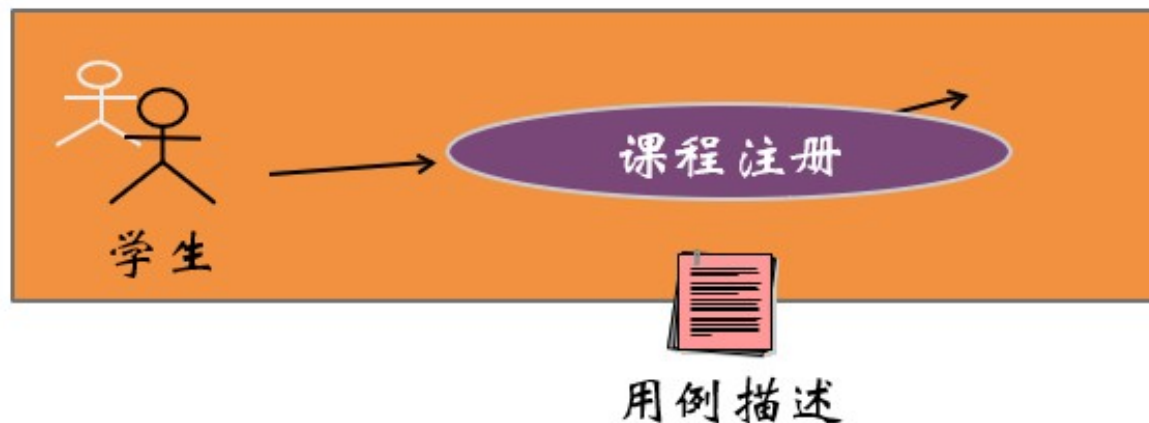
名称

学生

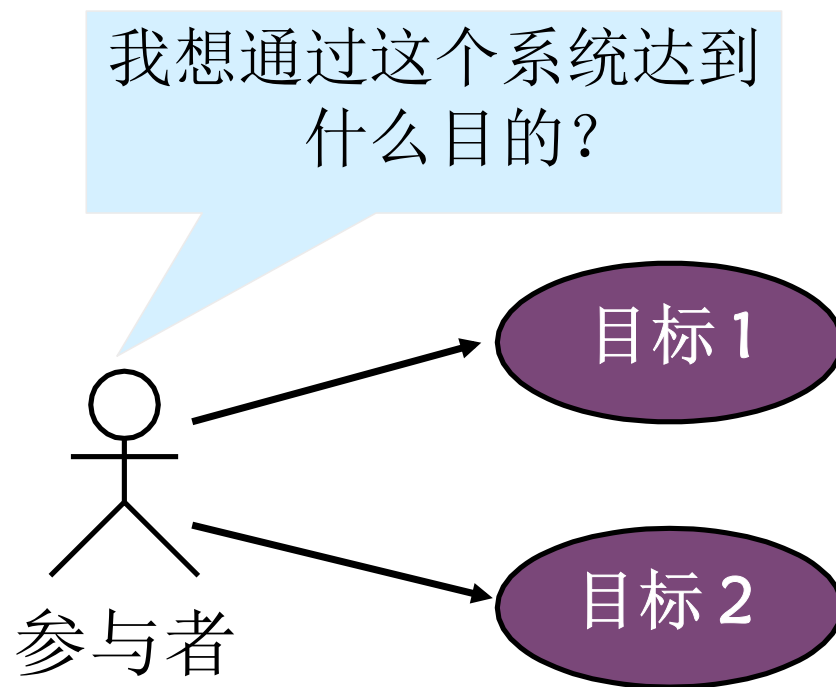
简要描述

注册课程的用户

和用例之间的关系



寻找用例



识别用例

- 每个参与者的目标是什么?
 - 为什么参与者要使用这个系统?
 - 参与者是否需要对系统中数据进行创建, 存储, 更改, 删除或者读取的操作? 为什么?
 - 参与者是否需要将外部事件或发生的改变告知系统?
 - 参与者是否需要知道系统内部发生的事件或改变?
- 系统是否能够应对业务中所有的正确行为与操作?

用例的描述

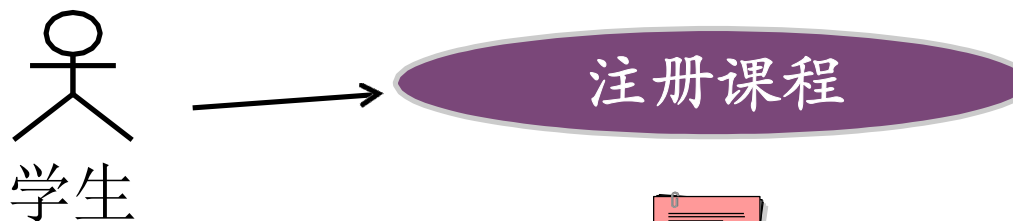
名称

注册课程

简要描述

学生选择下个学期想上的课程。生成必修课和选修课的课表信息。

与参与者的关系



用例描述

用例的命名

- 表明参与者的目标或者系统作用
- 使用主动语态：用动词起始
- 设计一系列操作流程(to-do list)

用例建模过程中的检查项

- 用例建模是为了表示系统的行为。通过模型可以很容易理解系统进行的操作
- 应该识别出所有的用例，用来表达所有的需求。
- 系统的任何一个特性都可以找到对应的用例
- 用例模型并不包含多余的行为；所有的用例可以追溯到系统的功能性需求作为验证。
- 去掉所有的**CRUD 类的用例**
 - **创建(Create)**, 查找(Retrieve), 更新(Update), 删除(Delete)



构建用例模型的步骤

- 第一步：找到所有的参与者和用例
- 识别出参与者并做简单的描述
 - 识别出用例并做简单的介绍
- 第二步：编写用例
 - 找出用例
 - 给用例事件流程划分重要等级
 - 按照重要程度排序
 - 详细描述事件流程



用例建模的过程：用例图→用例提纲→用例详细规约



+ 用例简单描述



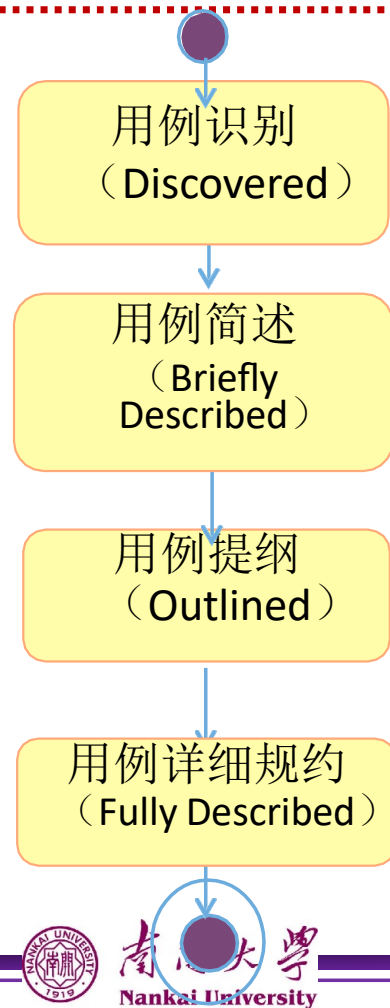
注册课程用例提纲
+ 粗略列出事件流程
• 大体步骤



注册课程用例的详细规约
+ 列出详细的事件流程
• 按步骤（详细）
+ 特殊的规约说明
+ 前置/后置条件



用例的全生命周期



结束课程注册

简述: 教务人员可以通过这个用例结束课程注册环节。学生人数不足的课程将被取消。收费系统会通知所有未被取消 的课程选课学生进行缴费。



结束注册环节 (概述)
-事件流
-...



结束注册环节 (用例规约)
-详细的事件流特
殊的需求
-前置/后置条件



用例简述的例子

- 用例简述:

一段简洁的摘要，主要描述用例的成功场景

- 处理购物交易:

客户带着要购买的货物到收款处，收银员使用POS机扫描记录每一种预购买的货物。系统计算总价并打印清单。客户付款，系统验证并保存销售记录。系统更新库存，客户得到收条并带着货物离开。

用例概述的例子

- 用例概述:
 - 非正式、随意的格式
 - 非正式段落，覆盖各种场景

退货处理

主成功场景:

客户带着要退的货物到达收款处，出纳员使用POS系统记录每一个要退 货的货物, ...

候选场景:

若信用验证失败，通知客户并要求使用其他付款方法
若系统检测到与外界计税系统通信失败, ...

详细用例规约的例子

- 用例名称: 下订单 (Place Order)

- 前置条件: 用户通过身份认证登录系统

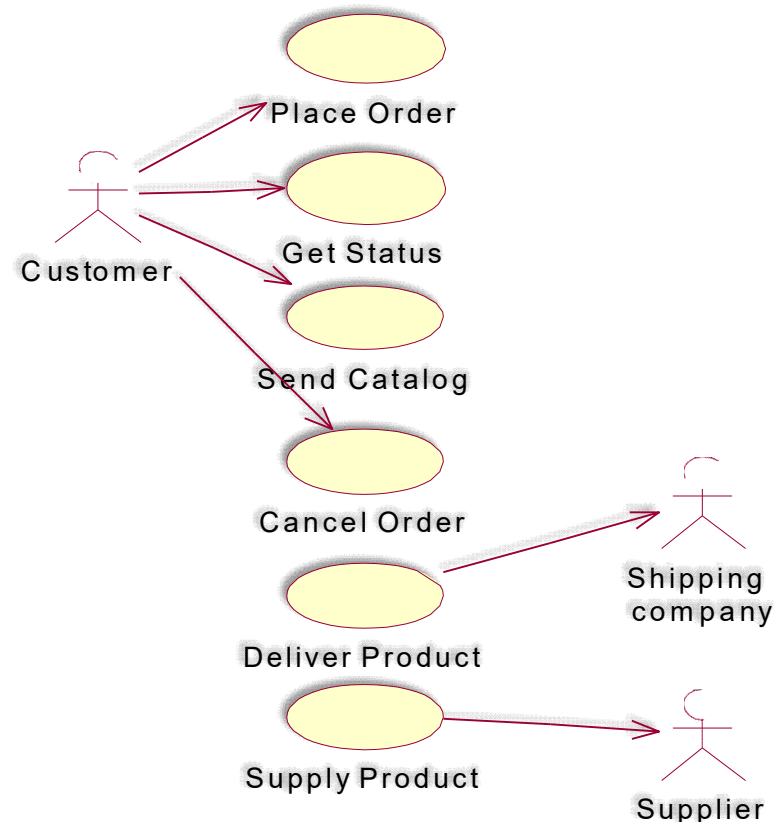
- 描述:

1. 当顾客选择“下订单”时, 进入该用例流程
2. 顾客输入姓名和地址信息
3. 如果顾客仅输入了邮政编码, 系统会提供州和城市信息
4. 顾客输入代购买的物品编码
5. 系统显示每个产品的描述信息和价格信息
6. 系统将持续记录顾客输入的所有商品信息和相应的总价
7. 顾客输入信用卡付账信息
8. 顾客选择提交(Submit)
9. 系统确认信息, 保存待付款订单信息, 将账单信息提交给账务系统
10. 确认付账成功后, 系统标记账单为完成状态, 向顾客显示账单ID信息, 用例结束 异常情况。

第9步中, 如果信息不正确, 系统将提示顾客对相应信息进行修改

后置条件: 系统保存订单并且标记为已确认。

用例图



用例文档模板

UC_id: 用例名

描述: 对该用例的一句或两句的描述。 **参与者:** 参与该用例的参与者。

包含: 该用例所包含的用例，以及包含它的用例。

扩展: 该用例可以扩展的用例，以及扩展它的用例。 **泛化:** 若该用例的子用例和父用例。

前置条件: 启动此用例所必须具备的条件。 **细节:** 该用例的细节。（基本流与可选流）

后置条件: 在该用例结束时确保成立的条件。

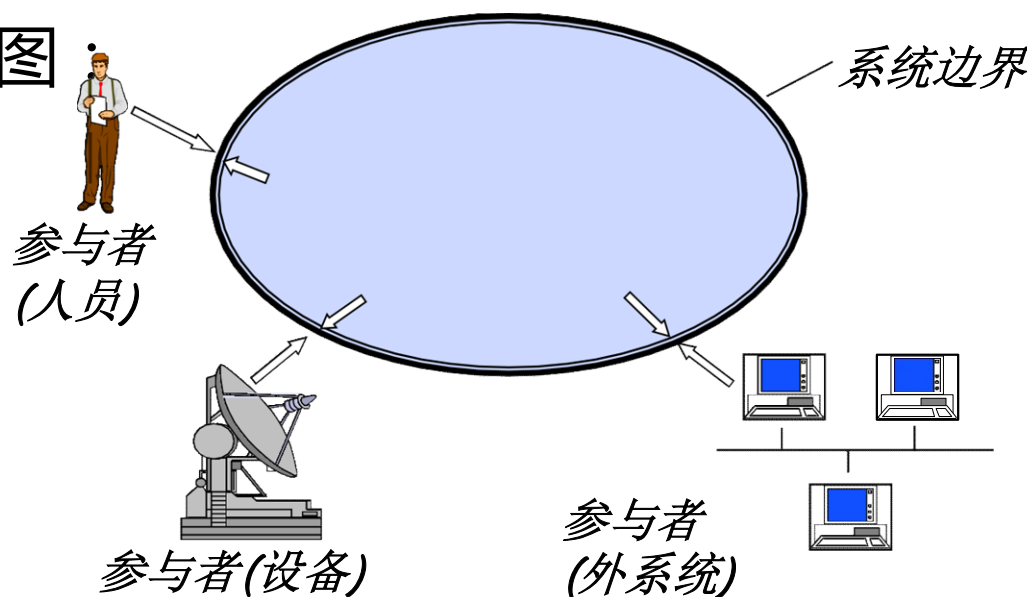
例外: 在该用例的执行的过程中可能引起的例外*。

限制: 在应用中可能出现的任何限制*。

注释: 提供可能对该用例是重要的任何附加信息。

一、设定系统边界

- **系统边界:** 一个系统所包含的所有系统成分与系统以外各种事物的分界线
- 系统边界会对用例以及Actor的定义有所影响
- 考虑用于零售店销售管理的系统的用例图：
 - 记录销售及付款情况的软硬件集成系统
 - 包括硬件设备，如计算机、条码扫描装置
 - 包括运行在系统上的软件
 - 系统目标包括：
 - 自动收款
 - 快速准确的销售情况统计及分析
 - 自动的库存管理

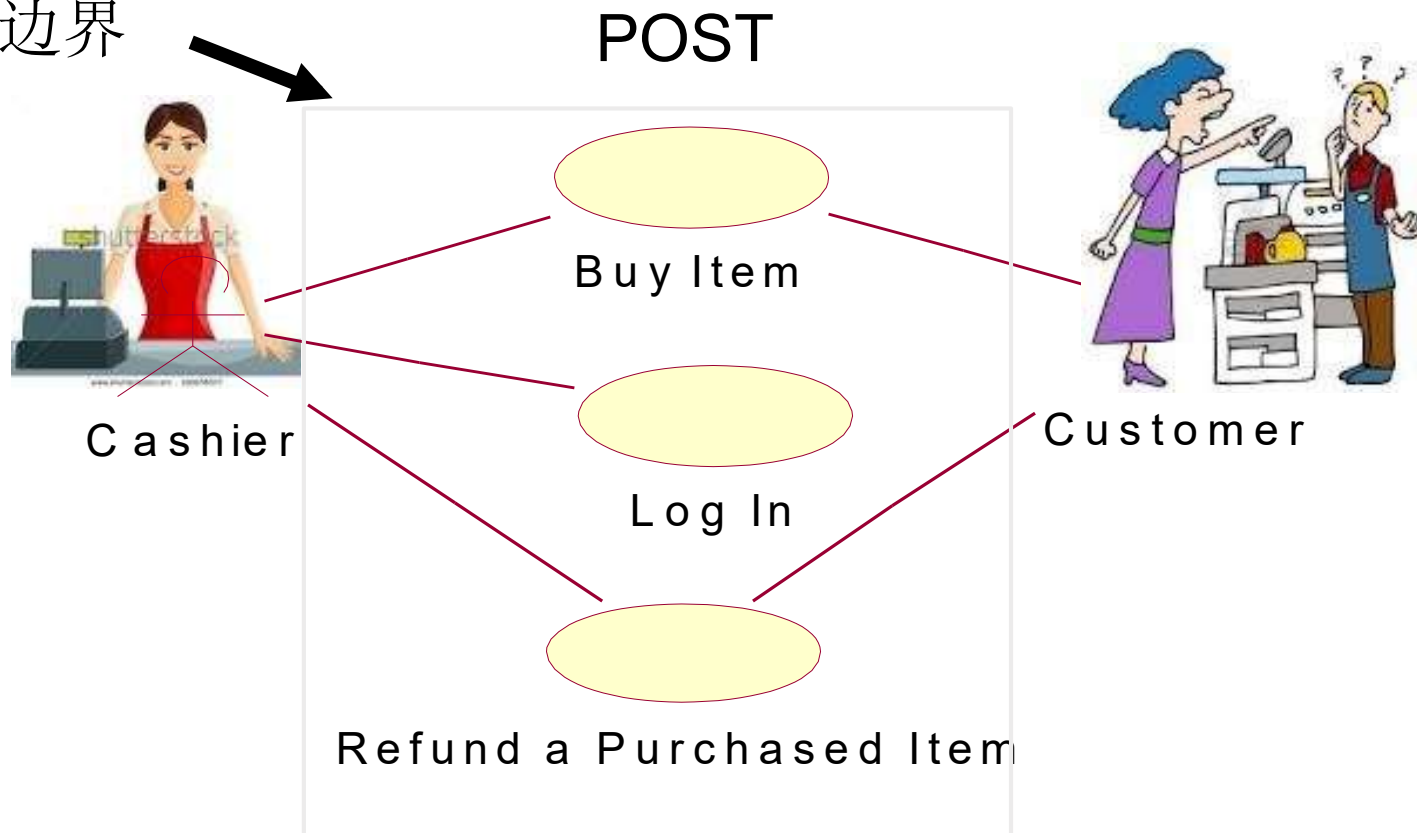


(本图选择自北大大麻志毅老师教材)



系统边界定义之一

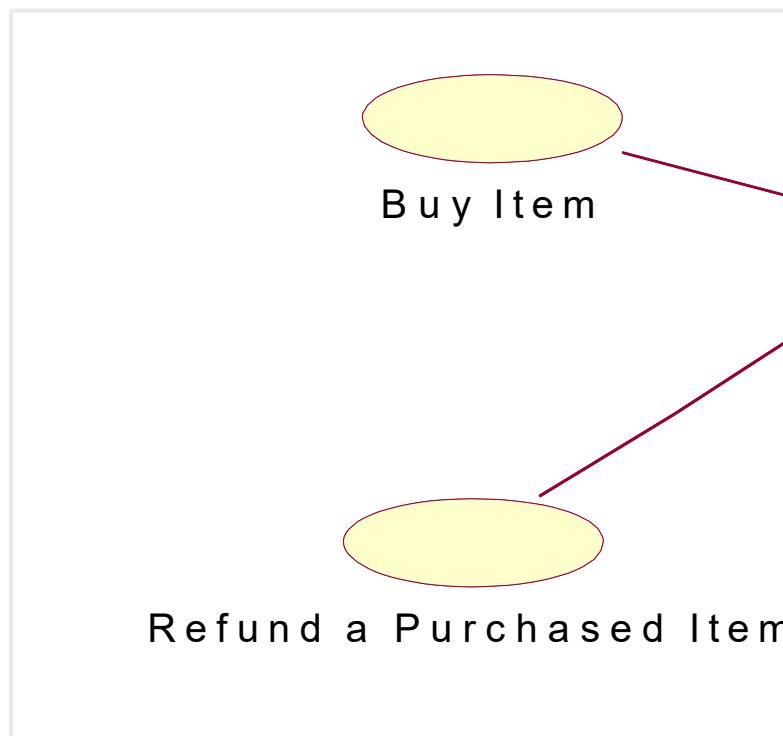
系统边界



系统边界定义之二

系统边界

POST

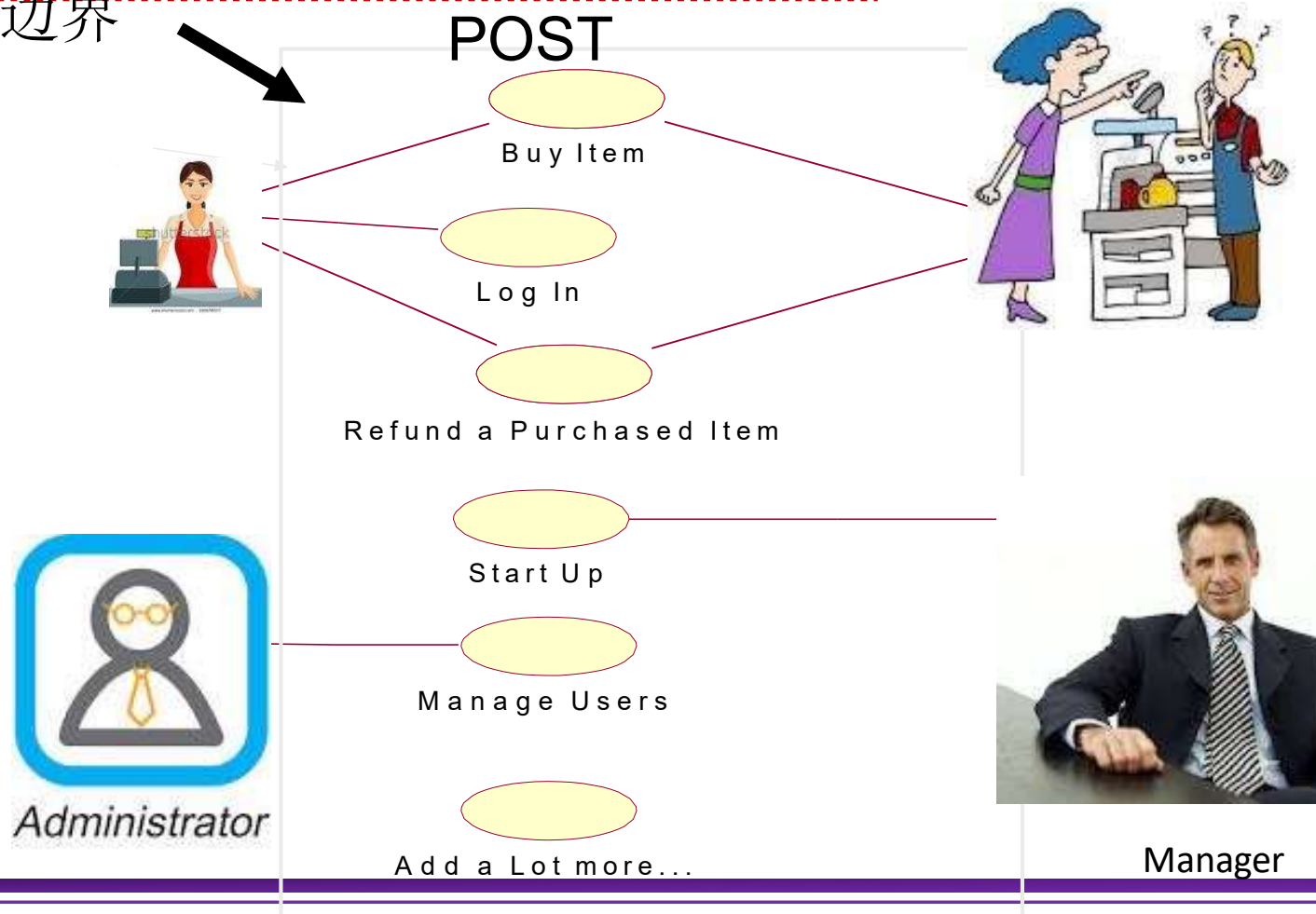


Customer



系统边界定义之三

系统边界

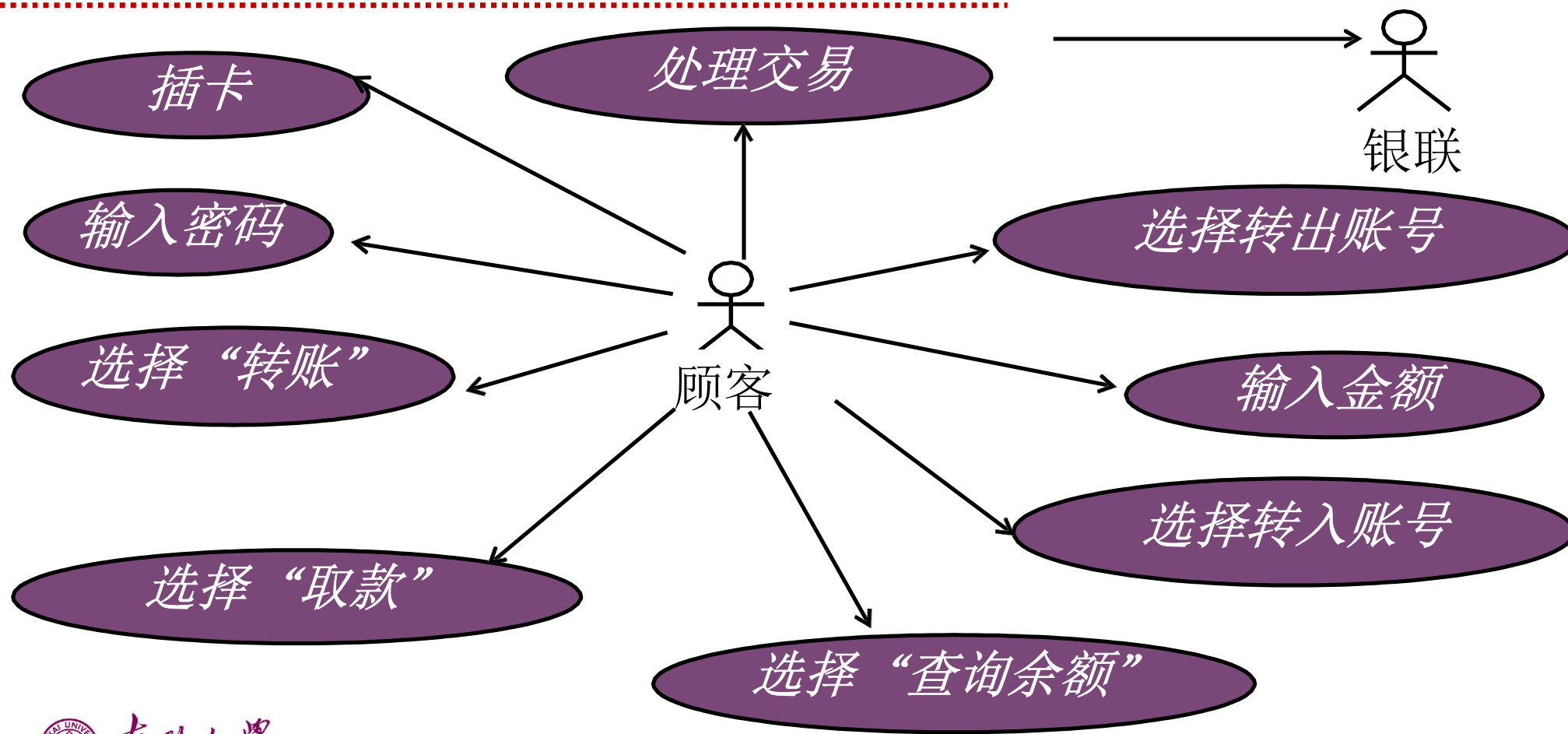


二、不要把用例定义成功能分解

- 功能分解：将问题分解为粒度小，独立的部分。
 - 不同的模块协同工作，体现系统的功能。
 - 通常，一些功能分解并没有实际的意义。 Often do not make sense in isolation.
- 用例：
 - **不是**功能分解的过程！
 - 综合所有功能一起描述系统如何使用。
 - 需要包含语境信息



功能分解：一个例子



走出功能分解：正确的用例建模



如何避免功能性分解？

问题现象

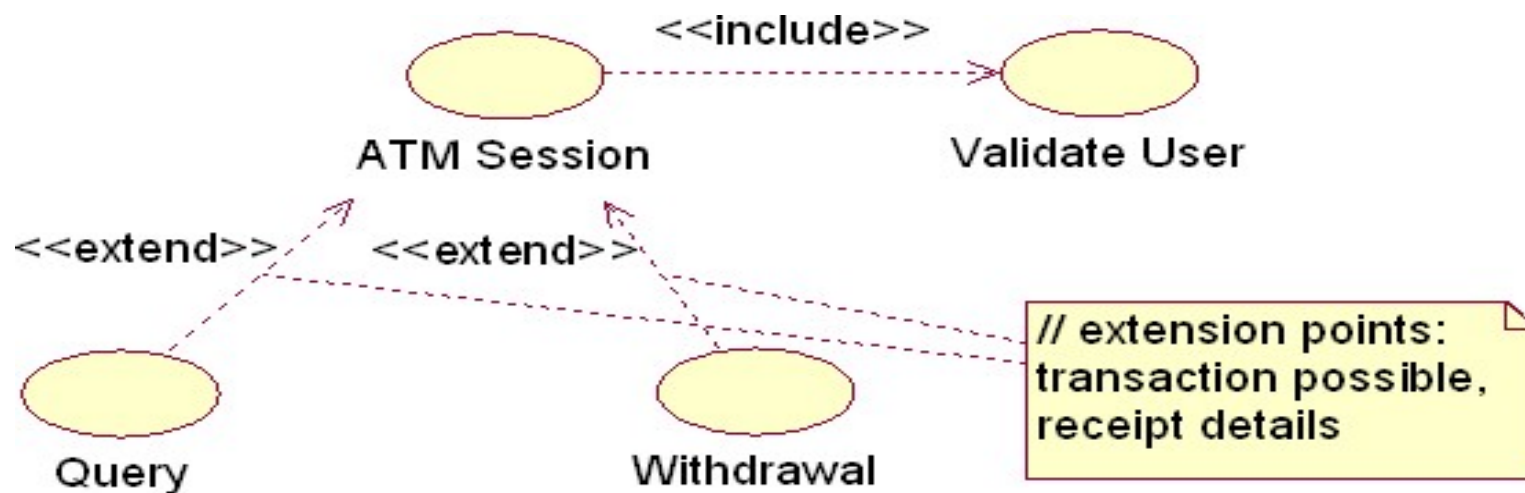
- 非常细小的用例
- 用例过多
- 没有实际价值的用例
- 通过底层操作进行命名
 - “操作” + “对象”
 - “功能” + “数据”
 - 例如：“插入卡片”
- 很难理解整体模型

修改思路：

- 寻找更大的应用场景
- “为什么要构建这个系统？”
- 从一个用户的角度出发
- “用户希望达到什么目的？”
- “这个用例可以满足谁的目标？”
- “这个用例的意义是什么？有什么价值？”
- “这个用例背后的用户故事是什么？”

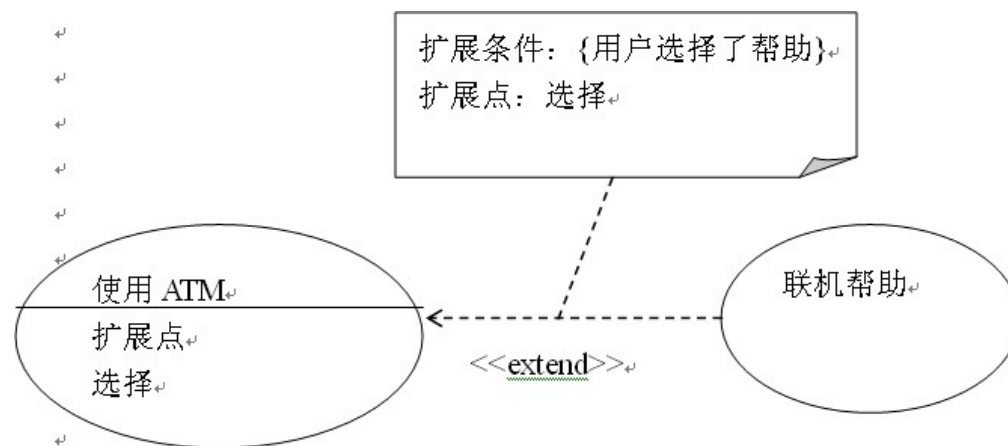
三、何时使用包含关系？

- 当多个用例有共享行为时，使用包含关系
- 为共享行为单独创建用例，被相关用例“包含”
- 例如：
 - 估价
 - 权限检查

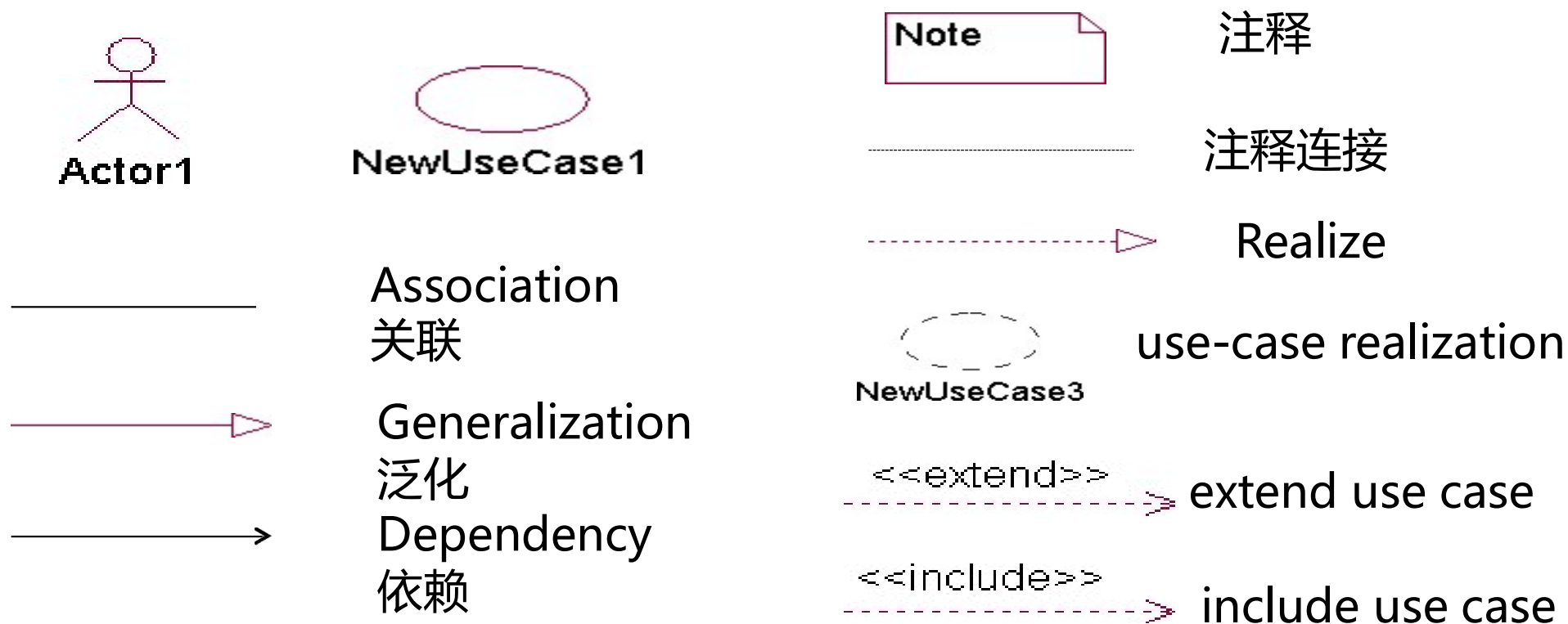


四、何时使用扩展关系？

- 一个用例与另外一个用例近似，只有少许额外的活动
- 将代表普遍或基本行为的情况定义为一个用例
- 将特殊的、例外的部分定义为扩展用例



五、用例图中的主要图标



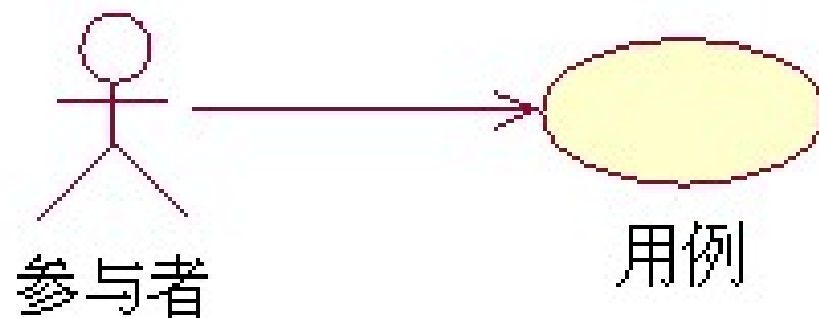
说明：UML中不使用颜色来作为图形语义的区分标记。

总结：Use Case模型的建立步骤

- (1) 找出系统外部的参与者和外部系统，确定系统的边界和范围；
- (2) 确定每一个参与者所期望的系统行为；
- (3) 把这些系统行为命名为Use Case；
- (4) 使用泛化、包含、扩展等关系处理系统行为的公共或变更部分；
- (5) 编制每一个Use Case的脚本；
- (6) 绘制Use Case图；
- (7) 区分主事件流和异常情况的事件流，可以把表示异常情况的事件流作为单独的Use Case处理；
- (8) 细化Use Case图，解决Use Case间的重复与冲突问题。

参与者与用例间的关联关系

关联关系，指参与者与用例之间的通信，也称为通信关系



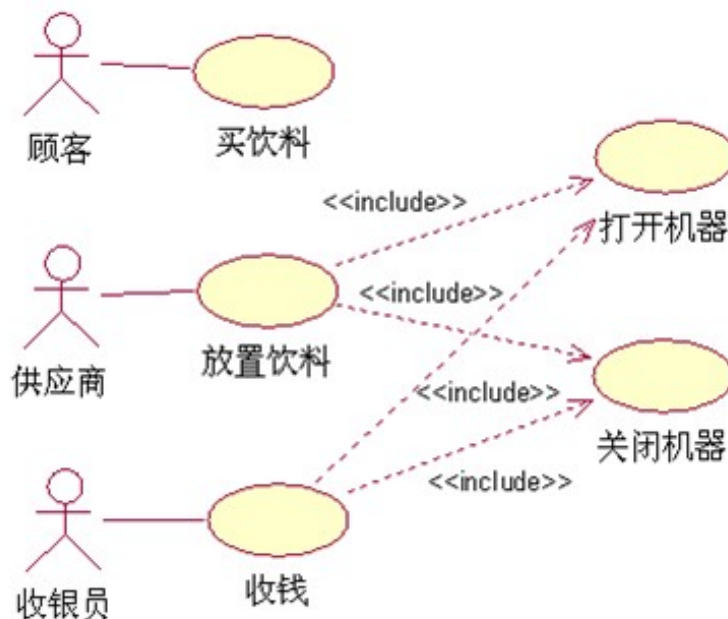
用例与用例的包含关系

包含关系描述的是一个用例需要某种功能，而该功能被另外一个用例定义，那么在用例的执行过程中，就可以调用已经定义好的用例。



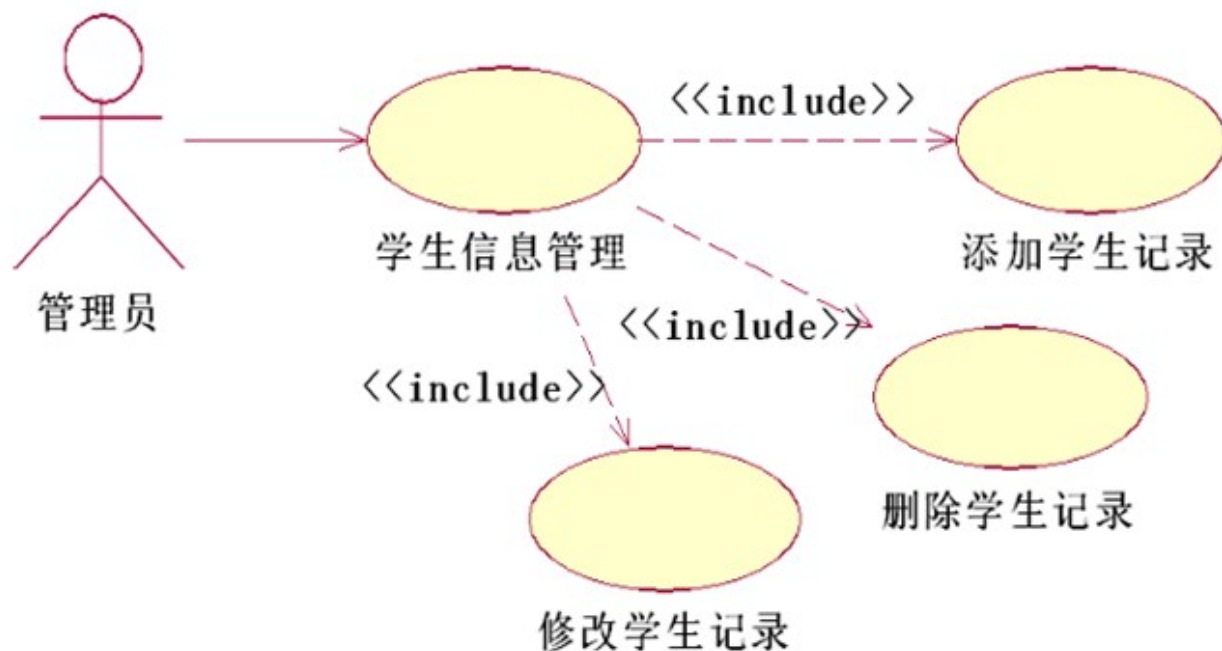
包含关系的使用场景一

如果**两个以上用例有大量一致的功能**，则可以将这个功能分解到另一个用例中，其他用例可以和这个用例建立包含关系



包含关系的使用场景二

一个用例的功能太多时，可以使用包含关系建立若干个更小的用例。



包含关系的特点

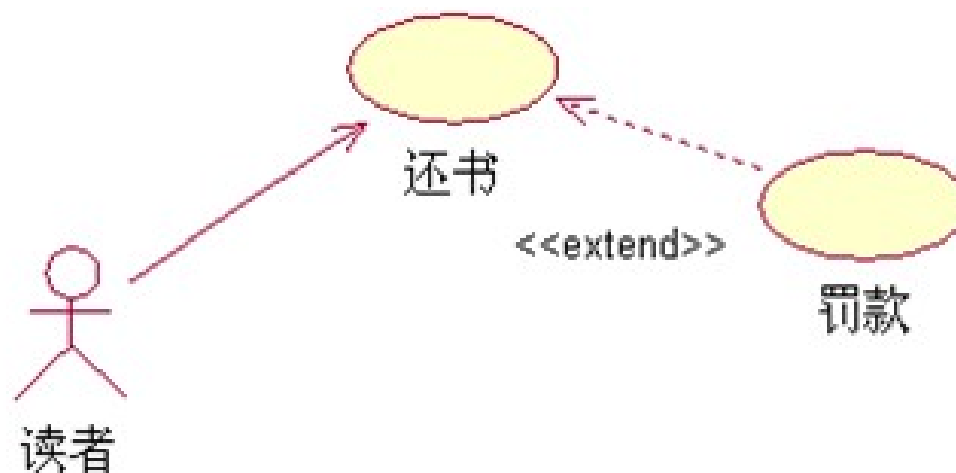
- 执行基用例时，每次都**必须调用**被包含用例。



- 包含关系有利于在系统建模阶段提前设计和规划有助于将来实现系统时，确定哪些功能可以**重用**，在编写代码时就可以实现代码的重用，缩短开发周期。

用例与用例的扩展关系

- 当某个新用例在原来的用例基础上增加了新的步骤序列，则原来用例被称为基用例，这种关系称为扩展关系。

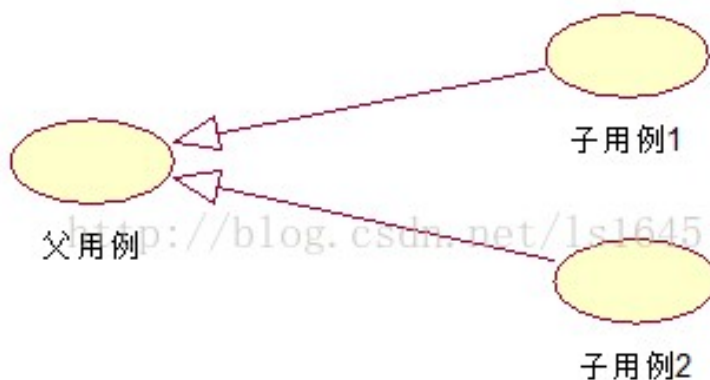


扩展关系的特性

- 只有当扩展点被激活时，子用例才会被执行。
- 适合将一些常规的动作放在一个基本用例中，将可选的或只在特定条件下才执行的动作放在它的扩展用例中。

用例和用例之间的泛化关系

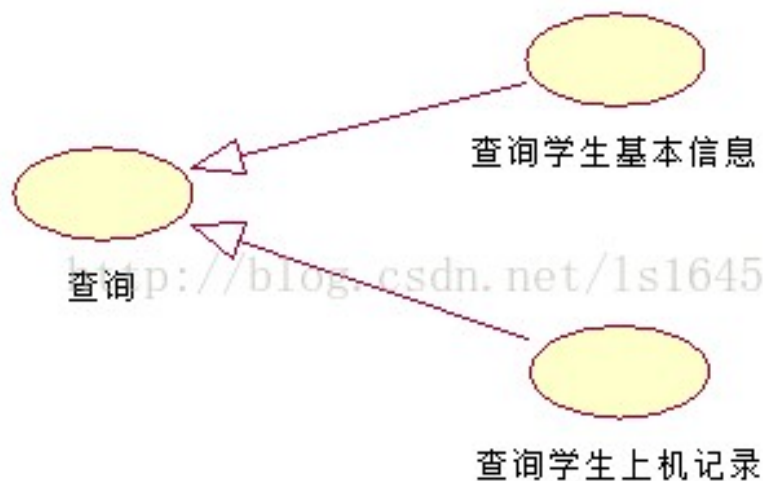
- 由具体的、个别的扩大为一般的被称为**泛化**。
- 子用例从父用例处继承行为和属性，还可以添加行为或覆盖、改变已继承的行为。



注意箭头方向！

泛化关系的特性

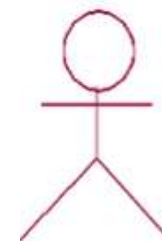
- 泛化关系在类中也有;
- 泛化和继承是不同的方向。泛化是从下到上的抽象过程，继承是从上到下，从一般到特殊的过程。
- 当系统中具有一个或多个用例是一般用例的特化时，就使用用例泛化。



1.识别参与者，确定系统的边界和范围

参与者：

- 系统外部的一个实体；
- 参与用例的执行过程；
- 通过向系统输入或请求系统输入某些事件来出发系统的执行；
- 由参与用例时的角色来表示；
- 每个参与者可以参与一个或多个用例；
- 参与者可以是系统用户，也可以是与系统交互的其它系统，还可以是一些可以运行的进程；



2.识别用例

- 用例是外部可见的系统单元;
- 可以通过确定每个参与者所期望的系统行为来识别用例;
- 用例名称用动宾结构表示。



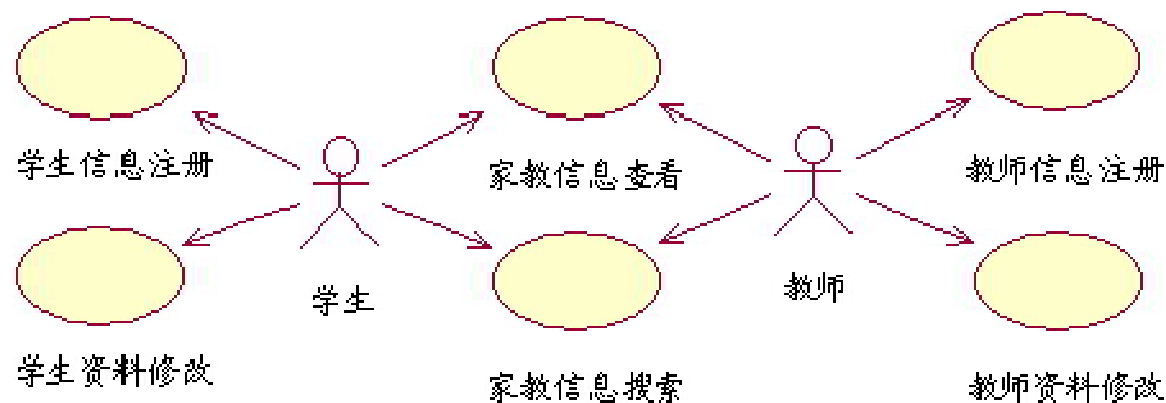
维护信息

3.对用例、参与者和它们之间的关系建模

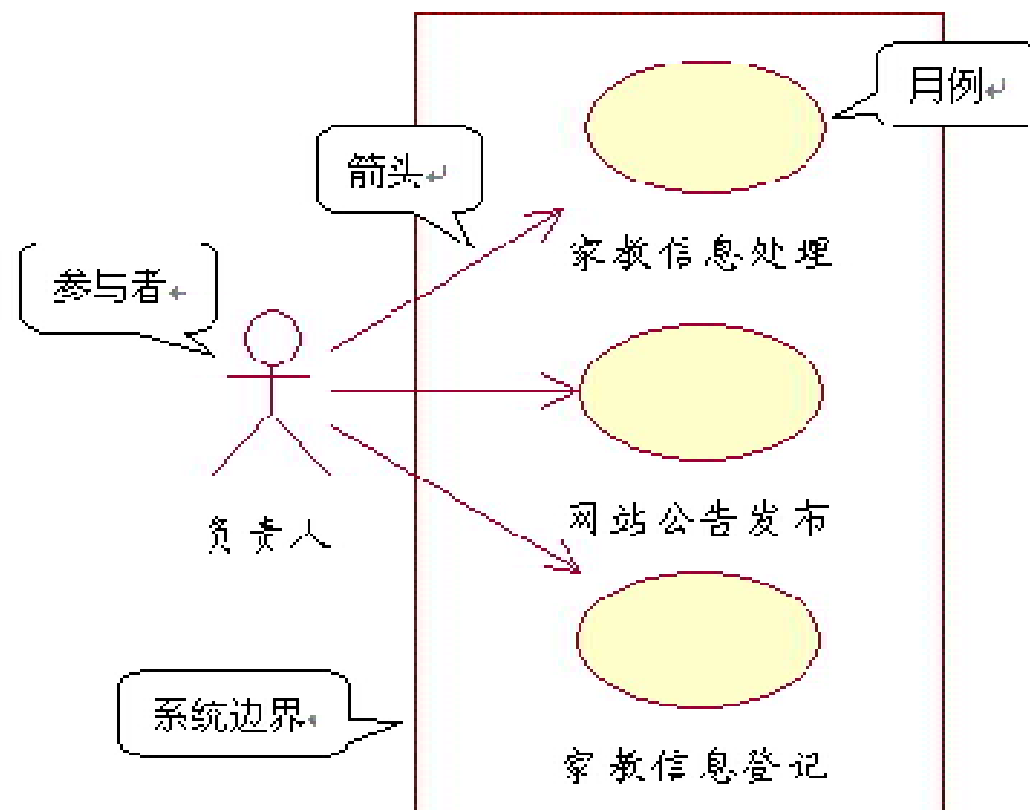
- 确定参与者与用例之间的通信路径—关联关系
- 确定用例之间的包含、扩展和泛化关系
- 绘制用例图，描述用例：
 - 简要说明
 - 前置条件
 - 事件流（主事件流、其它事件流、错误流）
 - 后置条件

用例图和用例描述设计实例--家教网站

- 前台客户系统的用例图：



- 后台管理系统用例图：



用例描述

用例名称: 网站公告发布
用例标识号: 202
参与者: 负责人
简要说明: 负责人用来填写和修改家教网站首页的公告, 公告最终显示在家教网站的首页上。
前置条件: 负责人已经登陆家教网站管理系统
基本事件流: 1. 负责人鼠标点击“修改公告”按钮 2. 系统出现一个文本框, 显示着原来的公告内容 3. 负责人可以在文本框上修改公告, 也可以完全删除, 重新写新的公告 4. 负责人编辑完文本框, 按“提交”按钮, 首页公告就被修改 5. 用例终止
其他事件流 A1: 在按“提交”按钮之前, 负责人随时可以按“返回”按钮, 文本框的任何修改内容都不会影响网站首页的公告
异常事件流: 1. 提示错误信息, 负责人确认 2. 返回到管理系统主页面
后置条件: 网站首页的公告信息被修改
注释: 无



统一建模语言UML

- UML (Unified Modeling Language)，是一种为面向对象系统的产品进行说明、可视化和编制文档的一种标准语言。
- 使用面向对象设计的建模工具，但独立于任何具体程序设计语言。
- UML有严格的语法和语义规范，采用一组图形符号来描述软件模型，可以直观地理解和阅读，由于具有规范性，所以能够保证模型的准确、一致。
- UML不仅可以用于软件建模，还可以用于其他领域的建模工作。

UML2.0

- 1.用例图：对系统的使用方式分类。
- 2.类图：显示类和它们的相互关系。
- 3.对象图：只显示对象及它们的相互关系。
- 4.活动图：显示人或对象的活动，其方式类似于流程图。
- 5.状态机图：显示生命周期比较有趣或复杂的对象的各种状态。
- 6.通信图：显示在某种情形下对象之间发送的消息。
- 7.顺序图：显示与通信图类似的信息，但强调的是顺序，而不是连接。

UML2.0

- 8.包图：显示相关的类如何组合，对开发人员有用。
- 9.部署图：显示安装已完成系统的机器、过程和部署制品。
- 10.组件图显示可重用的组件（对象或子系统）及其接口。
- 11.交互总图：活动图的一种形式，它的节点代表交互图。交互图包含顺序图，通信图，交互概览图和时间图。。
- 12.时间图：显示消息和对象状态的准确时间限制。
- 13.复合结构图：显示对象在聚合或复合中的相互关系，显示接口和协作的对象

UML的作用

- UML为软件系统建立可视化模型，便于直观理解和沟通。
- UML为软件系统建立构件，它的模型可以直接对应到各种各样的编程语言。
- UML为软件系统建立文档，不同的UML模型图可以作为项目不同阶段的软件开发文档。

系统建模工具的主要功能

- 可视化模型表达

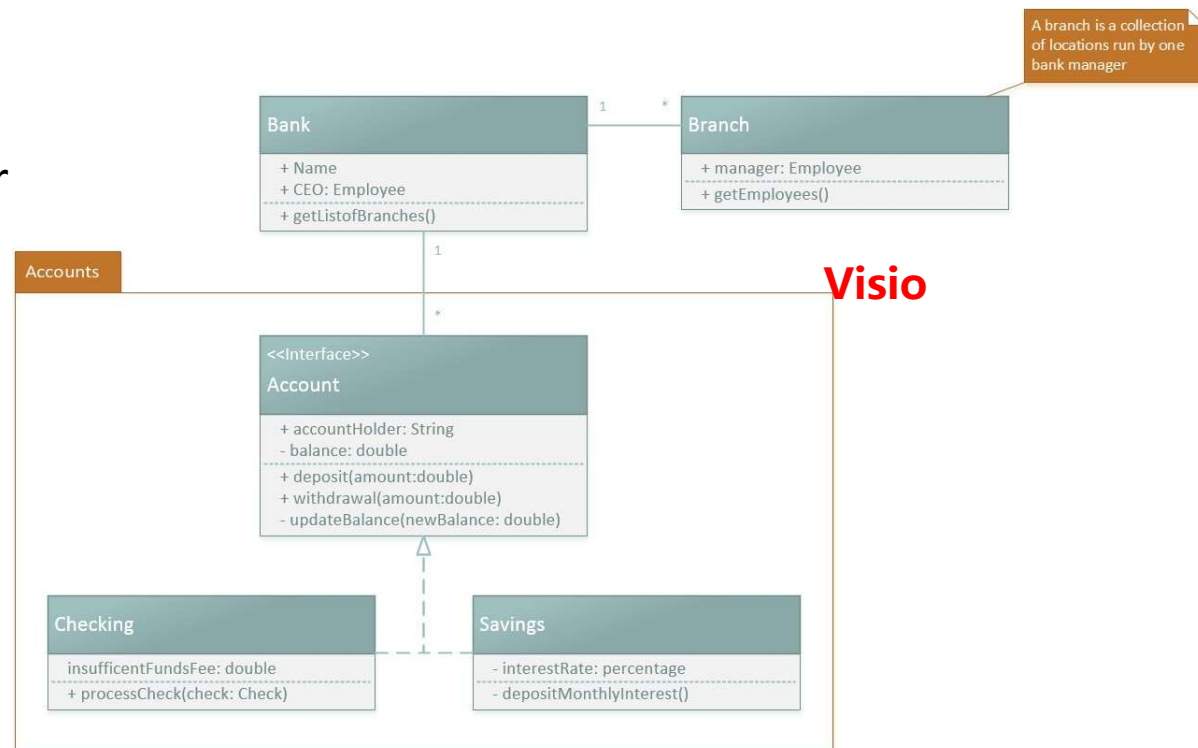
- UML模型
- Web模型, 例如Axure
- 数据库模型, 例如Power Designer
- 用户自定义模型, 例如Visio

- 画图工具

StarUML



- 辅助开发流程中的项目管理



常用的系统建模工具(UML2.0)

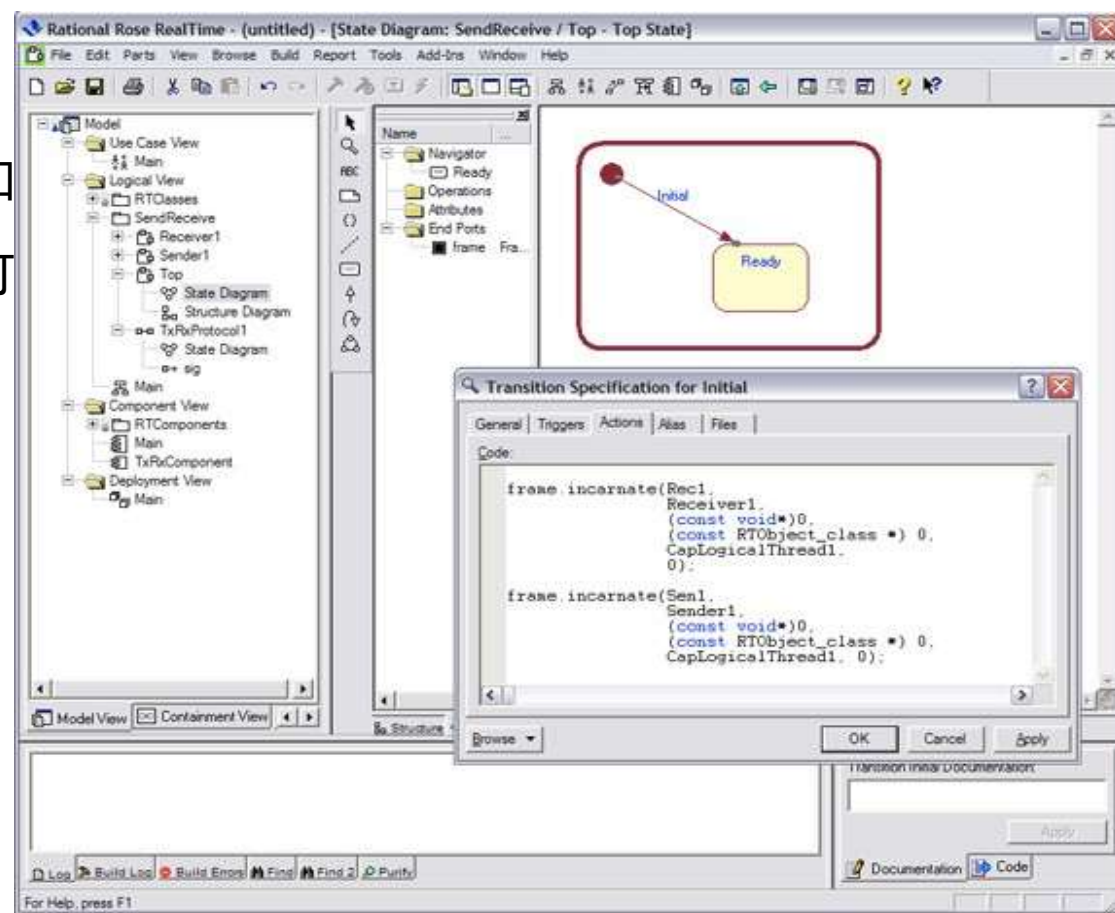
- IBM Rational Rose

- Rational是IBM设计的集成设计、建模和开发软件应用程序套件。Rose是其中的可视化建模工具

- 适用于应用程序开发、数据建模、Web Service设计、业务建模、组件建模等

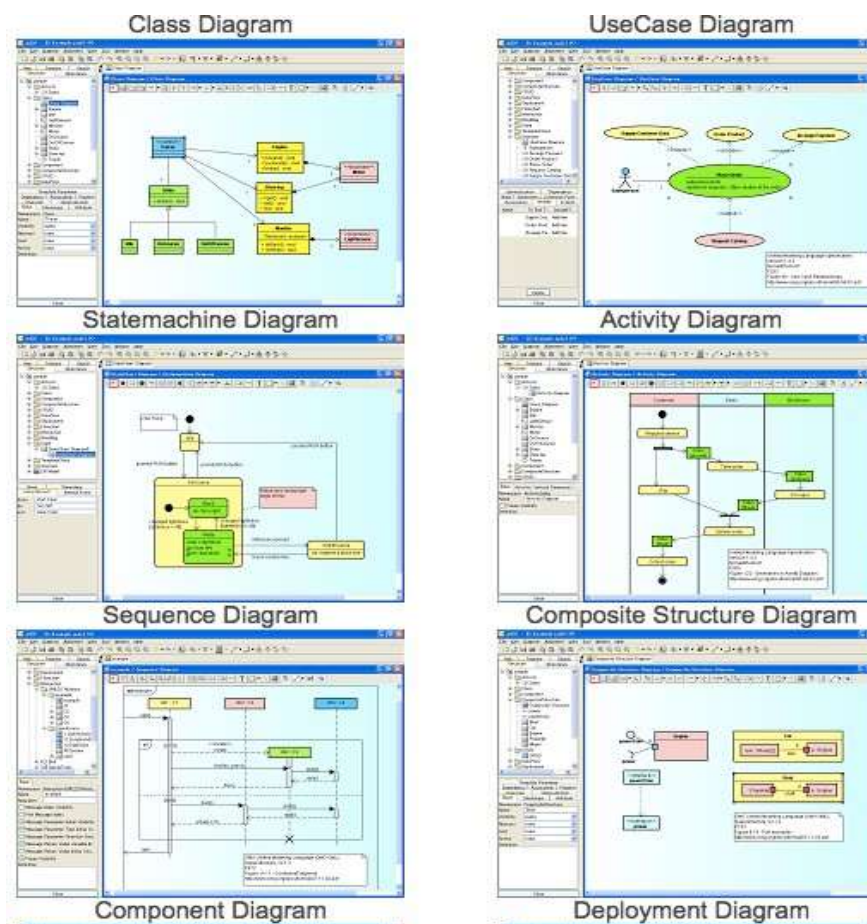
- 支持模型与代码之间的转化，逆向工程

资源链接: <http://www.ibm.com/developerworks/cn/rational/uml>



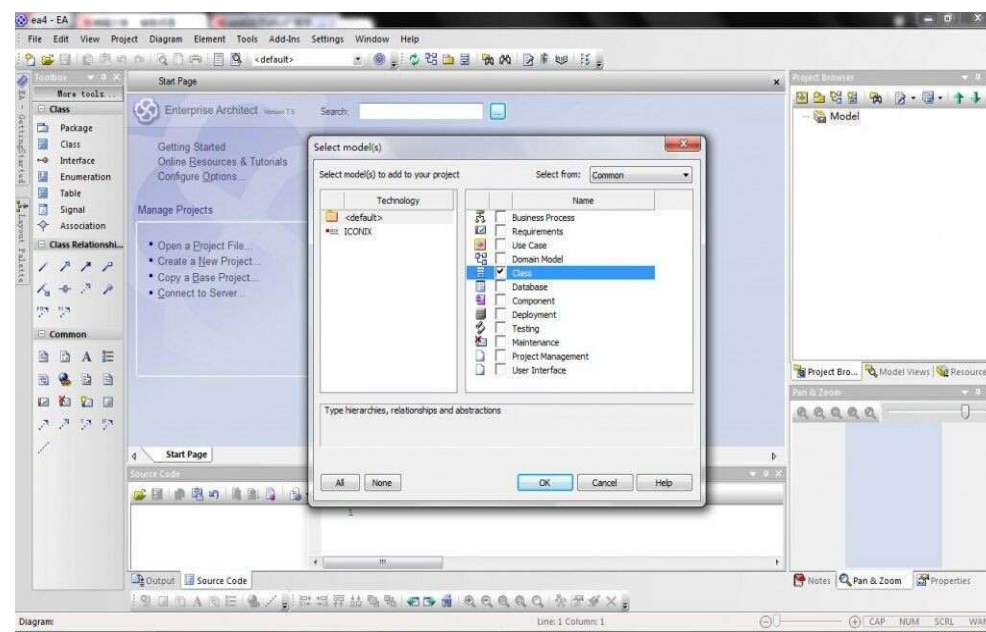
常用的系统建模工具(UML2.0)

- JUDE
 - 轻量级的UML建模工具
 - 支持项目团队中的协同工作
 - 文档转换
 - 全面支持UML1.4，支持UML2.0中部分模型



常用的系统建模工具(UML2.0)

- Enterprise Architect (EA)
 - 全功能的，基于UML2.0的可视化工具
 - 用于设计、编写、构建并管理以目标为导向的软件系统。
 - 为整个团队提供系统开发不同阶段所需的信息建模，服务于不同身份的成员
 - 提供从需求分析、软件设计一直到执行和部署整个过程的全面可跟踪性
 - 支持多种语言的前向或逆向生成代码工程



资源链接: <https://sparxsystems.com/products/ea/trial/request.html>

以下哪种关于用户故事和用例描述的说法是不正确的？

提交

- A 用户故事用于敏捷过程；用例描述用于统一建模过程
- B 用户故事作为开发者与用户交互面对面交互时的提示；用例作为项目文档保存
- C 用户故事可以用于估算；用例描述则不能用于估算
- D 用户故事采用自然语言文本描述；用例主要采用图形化的模型表示

需求说明文档通常不会采用以下哪种方式组织撰写？

- A 用户手册
- B 用户故事
- C 用例模型
- D 测试用例

提交

用例图中，当一个用例只在一定条件下比另一个用例增加少数步骤时，用哪种关系建模最合适？

提交

- A extends
- B includes
- C uses
- D inherits



取消订单用例与查询订单用例建模为以下哪种关系最合适?

提交

- A 关联关系
- B 依赖关系
- C 包含关系
- D 扩展关系

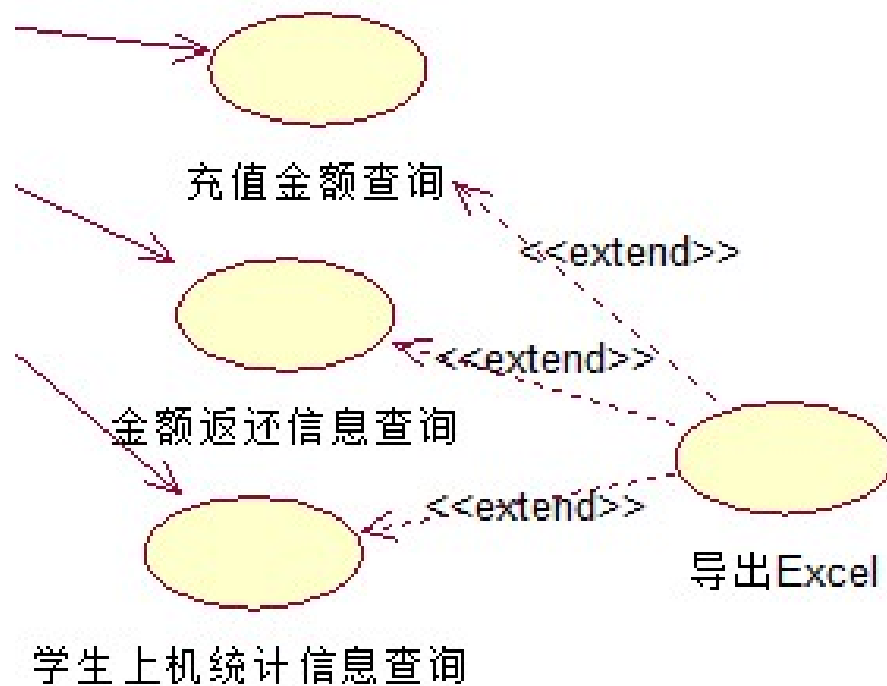


扩展关系 (Extend) : 这里的基用例是一个完整的用例，即使没有子用例的参与，也可以完成一个完整的功能，**只有当扩展点被激活时，子用例才会被执行**。由子用例指向基用例，比如说充值金额查询用例中有导出Excel子用例，离开子用例不影响充值金额查询的功能，这就是扩展关系。

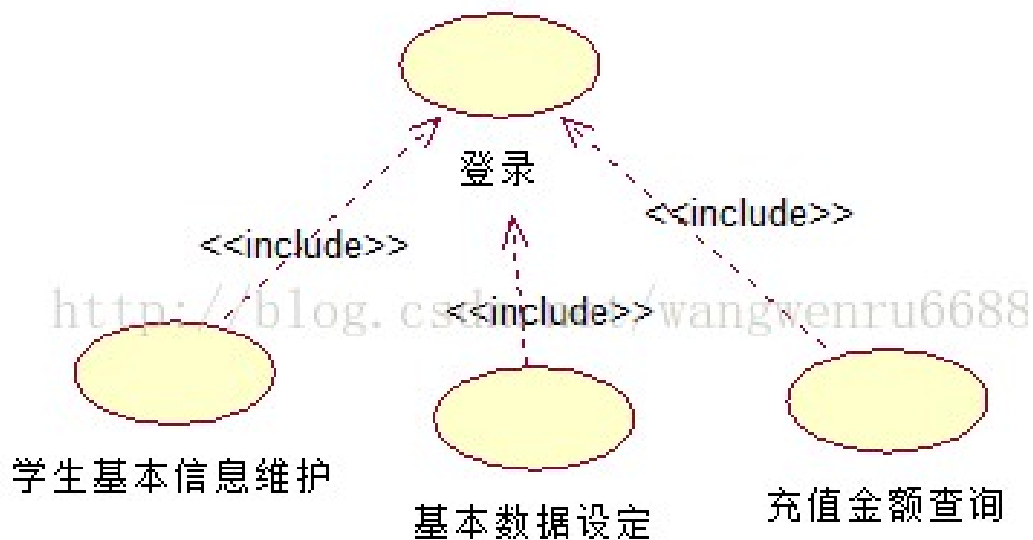
包含关系 (include) : 几个用例可以提取他们共用的用例作为子用例，使其成为自己行为的一部分，因为子用例被提出，基用例并非一个完整的用例，所以include关系中的**基用例必须和子用例一起使用**才够完整，子用例也必然被执行。



扩展关系



包含关系



以下哪个关于用例建模的说法是正确的？

提交

- A 用例可以定义系统功能性需求的优先级
- B 用例建模是对系统进行功能分解的过程
- C 用例能够描述非功能性的需求
- D 用例的参与者只能是系统用户

可以选用以下哪种工具进行用例建模?

提交

- A Microsoft Project
- B Enterprise Architect
- C Enterprise Architecture
- D IBM Rational DOORS

我们在为一家互联网电商开发订单处理软件，该公司从供应商那里购买产品，然后销售给客户。这家公司在线发布商品目录，并将其推送给客户和其他感兴趣的人。

客户以提交商品列表并向电商付费的方式购买商品。电商填写帐单，并委托快递公司把商品运送到客户的地址。订单处理软件记录从收到订单直到商品被运送给客户的整个过程。电商将提供快捷的服务，以最快、最有效的方法来发送客户订购的产品。客户可以退货，但有时要付运费。

电商订单处理软件系统的参与者不包括：

提交

- A 网络电商
- B 客户
- C 其它感兴趣的人
- D 快递公司



在网络电商客户定单处理应用中与客户有关的用例不包括：

提交

- A 退货
- B 计算运费
- C 浏览商品
- D 订单查询

进入订购商品用例的前置条件是：

提交

- A 客户对商品感兴趣
- B 客户安装了与系统兼容的浏览器版本
- C 商品已经放入购物车
- D 客户通过合法账户登入系统

使用订单处理系统一段时间以后，电商希望增加一种功能——为老顾客提供折扣。以下哪种方法比较合适？

提交

- A 建立老顾客折扣新用例
- B 扩展订购商品用例
- C 在订购商品用例中包含老顾客提供折扣用例
- D 为订购商品用例建立两个子用例：普通顾客订购商品和老顾客订购商品